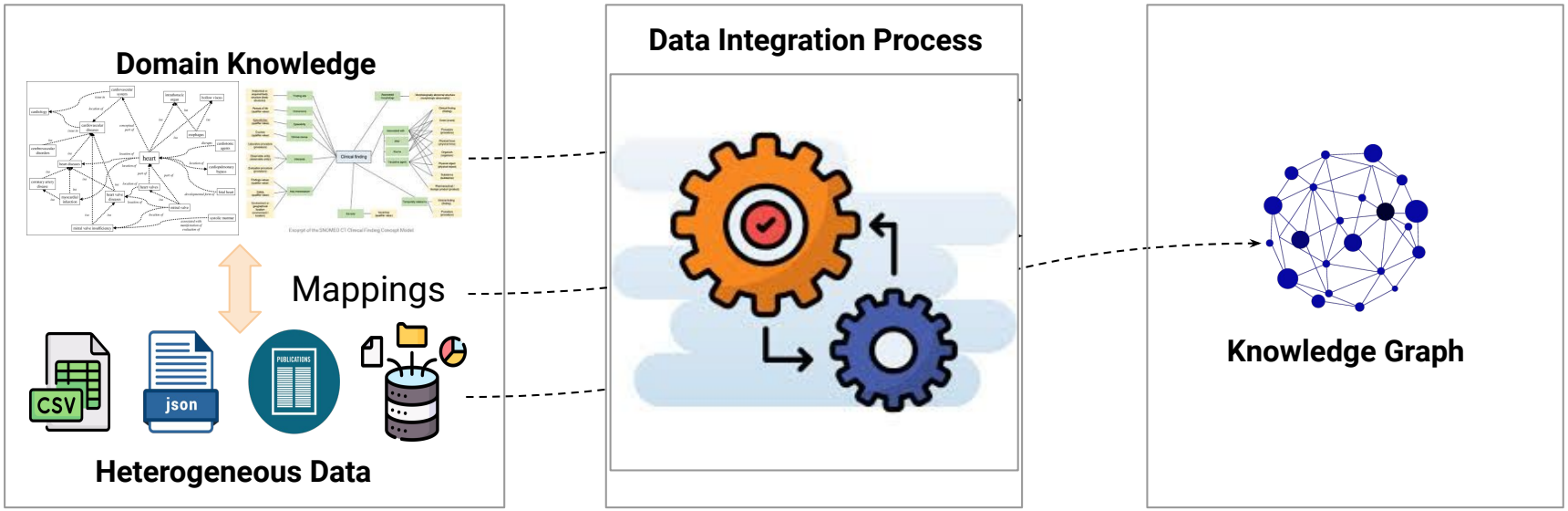
Knowledge Graph Creation & Enrichment (Lecture 9)

Knowledge Graph Creation & Enrichment (1/2)

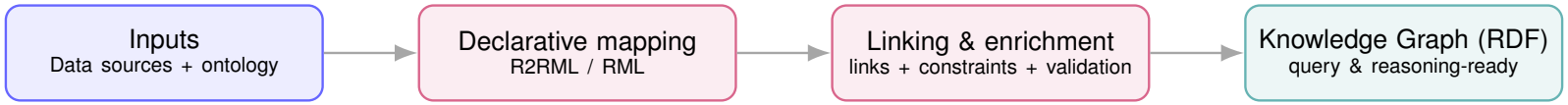


Diverse data sources (CSV, JSON, databases) and which can be mapped to ontologies to ensure semantic consistency in integration

Core components and rules of mapping languages like R2RML and RML, including mapping assertion rules for data integration

Knowledge Graphs represent integrated data for enhanced analytics and reasoning

Knowledge Graph Creation & Enrichment (2/2)



Examples:
CSV / JSON / RDB

Output:
RDF triples

Adds:
sameAs, SHACL checks

Supports:
SPARQL, analytics

Agenda

Part I — Foundations

- **Data Integration**
- Data Integration Systems

Part II — Mapping Languages

- Mapping Assertion Rules
- **R2RML** (Relational → RDF)
- **RML** (Heterogeneous Sources → RDF)

From heterogeneous data sources ⟶ RDF ⟶ Knowledge Graphs

Why Data Integration? [1]

What is Data Integration?

Data integration is a computational task in which data collected from multiple **autonomous data providers** is merged into a **unified data/knowledge base**.

The Core Problem

In practice, data is:

- **Distributed** across independent systems,
- **Heterogeneous** in structure, format, and semantics,
- **Evolving** independently over time.

Why Is This Hard?

Without integration:

- Users must understand each source individually,
- Queries must be rewritten for every data model,
- Results are incomplete, inconsistent, or incomparable.

Challenges of Data Integration

Query Processing

Challenge:
How to answer a single query over multiple, independent data sources?
Problem:

- Different query languages
- Different access interfaces
- No global schema

Heterogeneity

Challenge:
Sources are developed independently.
Conflicts include:

- Different schemata
- Different meanings of attributes
- Different data formats

Number of Sources

Challenge:
Integration complexity grows with each additional source.
Key insight:

- Hard even for two sources
- Mapping effort grows quickly

Autonomy

Challenge:
Sources may change without notice.

Implications:

- Schema evolution
- Changing access patterns
- No central control

Takeaway: Data integration is difficult not because of data volume, but because of **distribution, heterogeneity, and autonomy**.

Challenges of Data Integration

- **Query:** Processing queries over disparate data sources and offering a uniform view.
- **Number of sources:** The complexity of data integration depends on the number of data sources. However, data integration can be challenging even for two data sources.
- **Heterogeneity:** Data sources are developed independently. Data sources may suffer from various heterogeneity conflicts, e.g., different schemata, meaning of the attributes, and format.
- **Autonomy:** Data sources may change their data formats and access patterns at any time, without having to notify any central administrative entity.

What are Heterogeneity Conflicts?

Definition

Heterogeneity conflicts occur when autonomous data sources present incompatibilities that prevent seamless integration.

Causes

- Different domains of interpretation (semantic differences).
- Variations in schema structures (structural conflicts).
- Discrepancies in data values (data conflicts).
- Use of different representation languages (language conflicts).

Example of Heterogeneity Conflicts

Data Source 1: A relational database of clinical records (only part of the clinical records)

EHR	Diagnosis	Treatment	Doses	Start_Date	End_Date
555	Lung Cancer	Vinorelbina	3mg daily	30.10.22	01.11.2022
555	Lung Cancer	Cisplatino	10mg daily	30.10.22	01.11.2022
555	Lung Cancer	Omeprazol	75mg	30.10.22	01.11.2022

Patient_ID	Familial Antecedent	Type of Cancer
555	Mother	Breast
555	Father	Lung
555	Brother	Lung

Data Source 2: A relational database of publications about drugs/medicines

Drugs_mentioned_Publications

Publication ID	Drug
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI: 10.1155/2018/6278403	Vinorelbine
DOI:10.1159/000091889	Vinorelbine

Publication

Publication ID	Title	Authors	Journal	Year
DOI:10.2165/00003495-200767100-00003	Oral vinorelbine in the treatment of non-small cell lung cancer: rationale and implications for patient management	Richard J Gralla, Ulrich Gatzemeier, Vittorio Gebbia, Rudolf Huber, Mary O'Brien, Christian Puozzo	Drugs	2012
DOI:10.1159/000091889	Oral vinorelbine: pharmacology and treatment outcome in non-small cell bronchial carcinoma and breast carcinoma	Volker Bartsch	Onkologie	2006
DOI: 10.1155/2018/6278403	Metronomic Chemotherapy with Vinorelbine Produces Clinical Benefit and Low Toxicity in Frail Elderly Patients Affected by Advanced Non-Small Cell Lung Cancer	Michela D'Ascanio, Aldo Pezzuto, Chiara Fiorentino, Bruno Sposato, Pierdonato Bruno, Alessio Grieco, Rita Mancini, Alberto Ricci	Biomed Research Int	2018

Data Source 3: Data extracted from a web scientific database reporting interactions among drugs

Precipitant Drug	Object Drug	Effect	Impact
Vinorelbine	Cisplatin	Adverse effects	Increase
Omeprazole	Vinorelbine	Metabolism	Increase
Omeprazole	Cisplatin	Metabolism	Decrease

Data Source 4: Scientific database reporting interactions among drugs

Drug	Drug-Drug Interactions
DB00361	The combination of Vinorelbine with Cisplatin may increase the adverse effects
DB00515	The metabolism of Vinorelbine may be increased when combined with Omeprazole
DB00338	The metabolism of Omeprazole may be increased when combined with Cisplatin

Heterogeneity Conflicts (1/3)

- **Domain:** each source represents a different interpretation of a concept.

Heterogeneity Conflicts (1/3)

- **Domain:** each source represents a different interpretation of a concept.
  - Data Source 1 presents the drugs that composed the treatments administrated to a patient.
  - Data Source 2 comprises publications that describe a drug.
  - Data Source 3 presents the effects of the interactions between drugs.

Heterogeneity Conflicts (1/3)

- **Domain:** each source represents a different interpretation of a concept.
  - Data Source 1 presents the drugs that composed the treatments administrated to a patient.
  - Data Source 2 comprises publications that describe a drug.
  - Data Source 3 presents the effects of the interactions between drugs.

Data Source 1: A relational database of clinical records

EHR	Diagnosis	Treatment	Doses	Start_Date	End_Date
555	Lung Cancer	Vinorelbina	3mg daily	30.10.22	01.11.2022
555	Lung Cancer	Cisplatino	10mg daily	30.10.22	01.11.2022
555	Lung Cancer	Omeprazol	75mg	30.10.22	01.11.2022

Patient_ID	Familial Antecedent	Type of Cancer
555	Mother	Breast
555	Father	Lung
555	Brother	Lung

Data Source 2: A relational database of publications about drugs/medicines

Drugs_mentioned_Publications

Publication ID	Drug
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI: 10.1155/2018/6278403	Vinorelbine
DOI:10.1159/000091889	Vinorelbine

Publication

Publication ID	Title	Authors	Journal	Year
DOI:10.2165/00003495-200767100-00003	Oral vinorelbine in the treatment of non-small cell lung cancer: rationale and implications for patient management	Richard J Gralla, Ulrich Gatzemeier, Vittorio Gebbia, Rudolf Huber, Mary O'Brien, Christian Puozzo	Drugs	2012
DOI:10.1159/000091889	Oral vinorelbine: pharmacology and treatment outcome in non-small cell bronchial carcinoma and breast carcinoma	Volker Bartsch	Onkologie	2006
DOI: 10.1155/2018/6278403	Metronomic Chemotherapy with Vinorelbine Produces Clinical Benefit and Low Toxicity in Frail Elderly Patients Affected by Advanced Non-Small Cell Lung Cancer	Michela D'Ascanio, Aldo Pezzuto, Chiara Fiorentino, Bruno Sposato, Pierconato Bruno, Alessio Grieco, Rita Mancini, Alberto Ricci	Biomed Research Int	2018

Data Source 3: Data extracted from a web scientific database reporting interactions among drugs

Precipitant Drug	Object Drug	Effect	Impact
Vinorelbine	Cisplatin	Adverse effects	Increase
Omeprazole	Vinorelbine	Metabolism	Increase
Omeprazole	Cisplatin	Metabolism	Decrease

Heterogeneity Conflicts (2/3)

- **Structural Conflict:** This interoperability conflict exists between data sources modeled using different schemata.
  - The four data sources are modeled using different schemata and represented in two data models:
    - Relational models for Data Sources 1, 2, and 3.
    - Semi-structured model for Data Source 4.
  - The concept of drug interaction is represented:
    - As a single attribute in Data Source 4.
    - As three separate attributes in Data Source 3.
  - Different attribute names are used to represent a drug:
    - Treatment (Source 1).
    - Drug (Source 2).
    - Precipitant Drug and Object Drug (Source 3).

Data Source 1: A relational database of clinical records (only part of the clinical records)

EHR	Diagnosis	Treatment	Doses	Start_Date	End_Date
555	Lung Cancer	Vinorelbina	3mg daily	30.10.22	01.11.2022
555	Lung Cancer	Cisplatino	10mg daily	30.10.22	01.11.2022
555	Lung Cancer	Omeprazol	75mg	30.10.22	01.11.2022

Patient_ID	Familial Antecedent	Type of Cancer
555	Mother	Breast
555	Father	Lung
555	Brother	Lung

Data Source 2: A relational database of publications about drugs/medicines

Drugs_mentioned_Publications

Publication ID	Drug
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI: 10.1155/2018/8278403	Vinorelbine
DOI:10.1159/000091889	Vinorelbine

Publication

Publication ID	Title	Authors	Journal	Year
DOI:10.2165/00003495-200767100-00003	Oral vinorelbine in the treatment of non-small cell lung cancer: rationale and implications for patient management	Richard J Gralla, Ulrich Gatzemeier, Vittorio Gebbia, Rudolf Huber, Mary O'Brien, Christian Puozzo	Drugs	2012
DOI:10.1159/000091889	Oral vinorelbine: pharmacology and treatment outcome in non-small cell bronchial carcinoma and breast carcinoma	Volker Bartsch	Onkologie	2006
DOI: 10.1155/2018/8278403	Metronomic Chemotherapy with Vinorelbine Produces Clinical Benefit and Low Toxicity in Frail Elderly Patients Affected by Advanced Non-Small Cell Lung Cancer	Michela D'Ascanio, Aldo Pezzuto, Chiara Fiorentino, Bruno Sposato, Pierdonato Bruno, Alessio Greco, Rita Mancini, Alberto Ricci	Biomed Research Int	2018

Data Source 3: Data extracted from a web scientific database reporting interactions among drugs

Precipitant Drug	Object Drug	Effect	Impact
Vinorelbine	Cisplatin	Adverse effects	Increase
Omeprazole	Vinorelbine	Metabolism	Increase
Omeprazole	Cisplatin	Metabolism	Decrease

Data Source 4: Scientific database reporting interactions among drugs

Drug	Drug-Drug Interactions
DB00361	The combination of Vinorelbine with Cisplatin may increase the adverse effects
DB00515	The metabolism of Vinorelbine may be increased when combined with Omeprazole
DB00338	The metabolism of Omeprazole may be increased when combined with Cisplatin

Heterogeneity Conflicts (3/3)

- **Data:** This interoperability conflict arises when there are discrepancies among similar or related data values.
  - Data sources 1, 2, and 3 represent the drugs with their names while data source 4 uses DrugBank identifiers.
- **Language:** This interoperability conflict appears whenever different languages are used to represent the data or metadata, i.e., schema.
  - Drugs in data source 1 are in Spanish, while they are in English in the other data sources.

Data Source 1: A relational database of clinical records (only part of the clinical records)

EHR	Diagnosis	Treatment	Doses	Start_Date	End_Date
555	Lung Cancer	Vinorelbina	3mg daily	30.10.22	01.11.2022
555	Lung Cancer	Cisplatino	10mg daily	30.10.22	01.11.2022
555	Lung Cancer	Omeprazol	75mg	30.10.22	01.11.2022

Patient_ID	Familial Antecedent	Type of Cancer
555	Mother	Breast
555	Father	Lung
555	Brother	Lung

Data Source 2: A relational database of publications about drugs/medicines

Drugs_mentioned_Publications

Publication ID	Drug
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI:10.2165/00003495-200767100-00003	Vinorelbine
DOI: 10.1155/2018/8278403	Vinorelbine
DOI:10.1159/000091889	Vinorelbine

Publication

Publication ID	Title	Authors	Journal	Year
DOI:10.2165/00003495-200767100-00003	Oral vinorelbine in the treatment of non-small cell lung cancer: rationale and implications for patient management	Richard J Gralla, Ulrich Gatzemeier, Vittoro Gebbia, Rudolf Huber, Mary O'Brien, Christian Puozzo	Drugs	2012
DOI:10.1159/000091889	Oral vinorelbine: pharmacology and treatment outcome in non-small cell bronchial carcinoma and breast carcinoma	Volker Bartsch	Onkologie	2006
DOI: 10.1155/2018/8278403	Metronomic Chemotherapy with Vinorelbine Produces Clinical Benefit and Low Toxicity in Frail Elderly Patients Affected by Advanced Non-Small Cell Lung Cancer	Michela D'Ascanio, Aldo Pezzuto, Chiara Fiorentino, Bruno Sposato, Pierdonato Bruno, Alessio Greco, Rita Mancini, Alberto Ricci	Biomed Research Int	2018

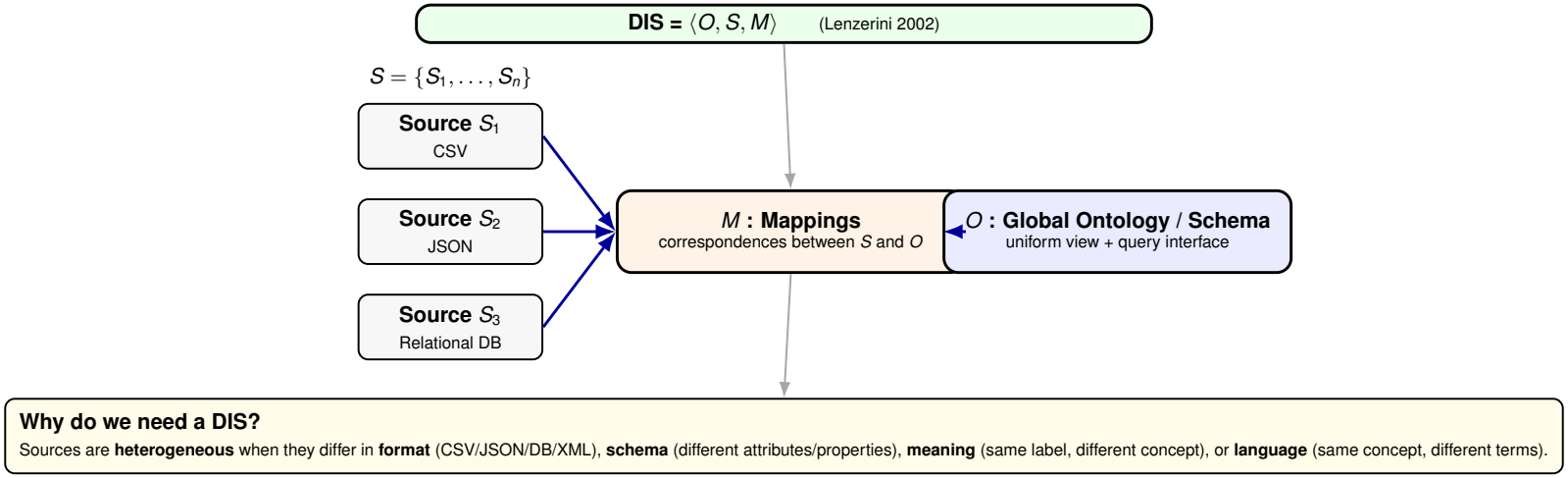
Data Source 3: Data extracted from a web scientific database reporting interactions among drugs

Precipitant Drug	Object Drug	Effect	Impact
Vinorelbine	Cisplatin	Adverse effects	Increase
Omeprazole	Vinorelbine	Metabolism	Increase
Omeprazole	Cisplatin	Metabolism	Decrease

Data Source 4: Scientific database reporting interactions among drugs

Drug	Drug-Drug Interactions
DB00361	The combination of Vinorelbine with Cisplatin may increase the adverse effects
DB00515	The metabolism of Vinorelbine may be increased when combined with Omeprazole
DB00338	The metabolism of Omeprazole may be increased when combined with Cisplatin

Data Integration System (DIS)



Intuition: users query O once; M connects O to all local sources S.

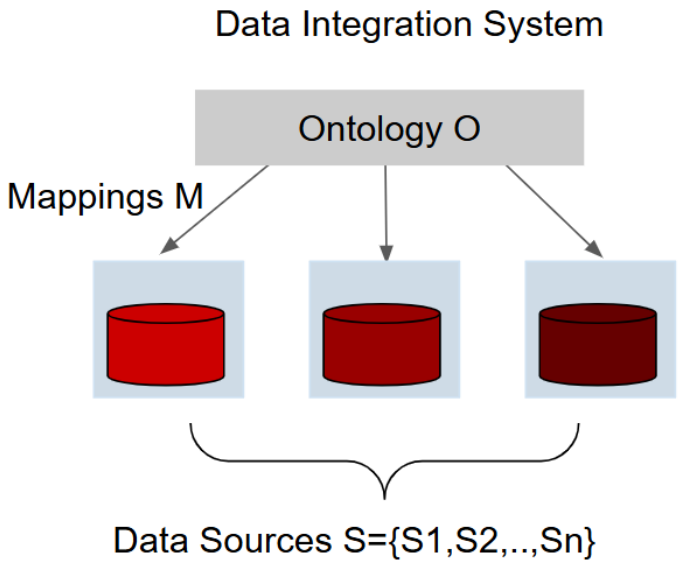
What is a Data Integration System (DIS)? (1/2)

Definition: A **Data Integration System (DIS)** is a triple [2]:

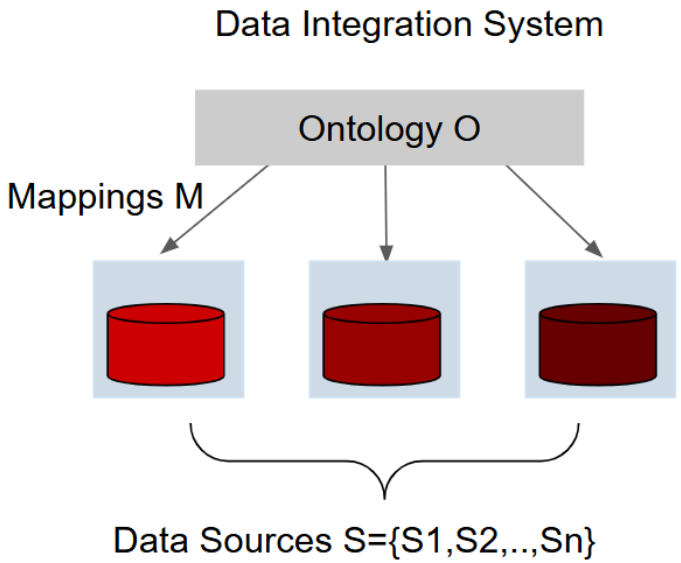
$$DIS = \langle O, S, M \rangle$$

- **O (Ontology or Schema):**
  - Provides a uniform view of the data sources.
  - Acts as the global schema to query the integrated data.
- **S (Signatures of Data Sources):**
  - A set $S = \{S_1, S_2, \ldots, S_n\}$ representing the local schemata or structures of the individual data sources.
- **M (Mappings):**
  - Defines correspondences between the global ontology/schema O and the local data source signatures in S.
  - Enables transformation of queries and data between the global and local layers.

What is a Data Integration System (DIS)? (2/2)

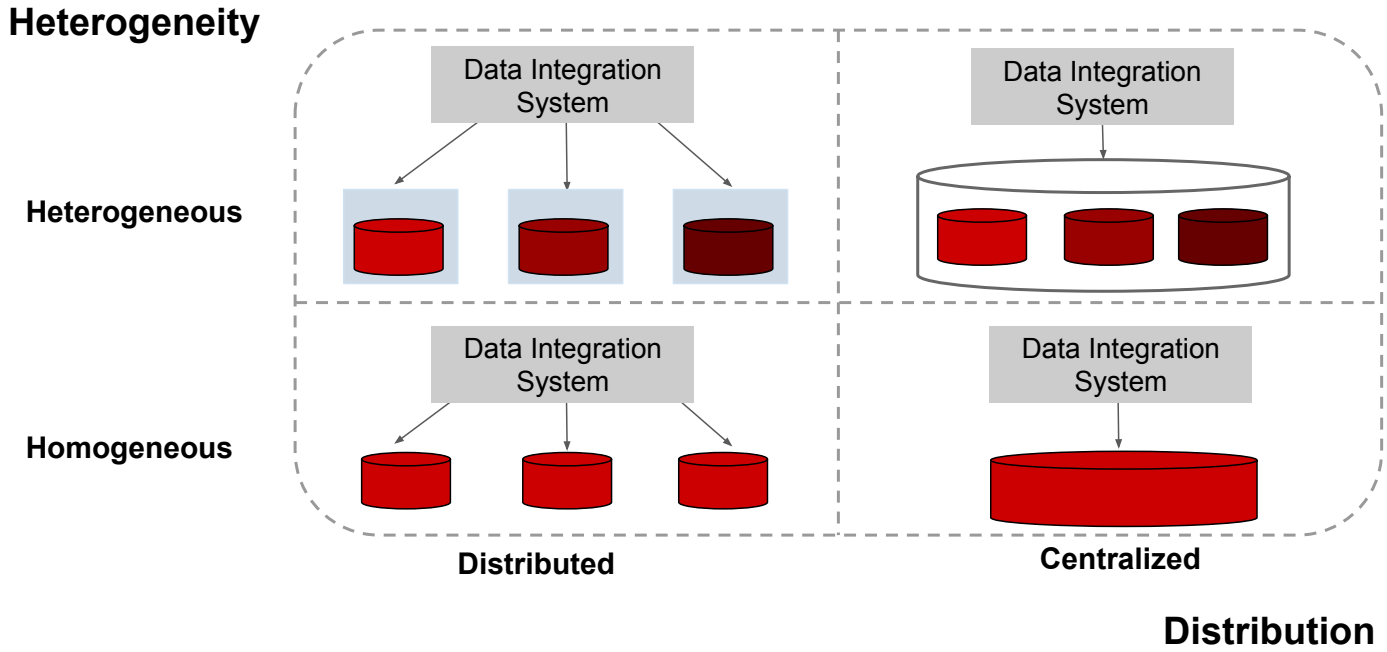


What is a Data Integration System (DIS)? (2/2)

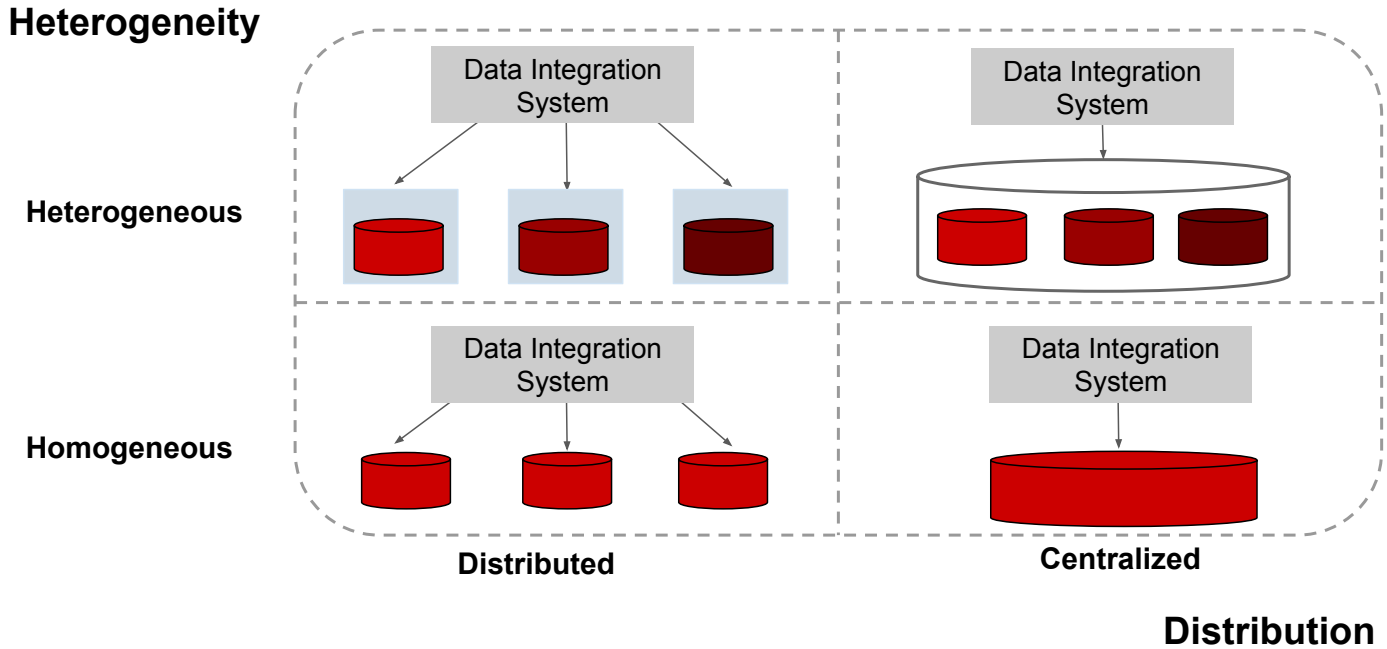


- Data Integration Systems are needed
  - to provide a uniform view, and to integrate data collected from heterogeneous data sources
- Two sources are heterogeneous (or have heterogeneity conflicts) when:
  - they are presented in different formats (e.g., CSV, JSON, Relational Tables, XML)
  - they represent different information about the same concept
  - they use different concepts (i.e., attributes, classes, properties) to represent the same concepts
  - they represent the same concepts in different languages

Types of Data Integration Systems (1/2)

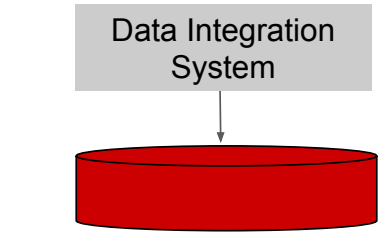
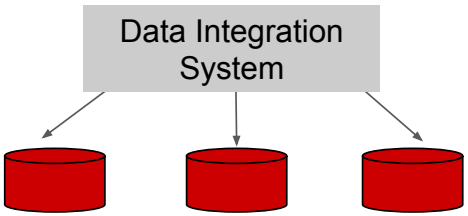
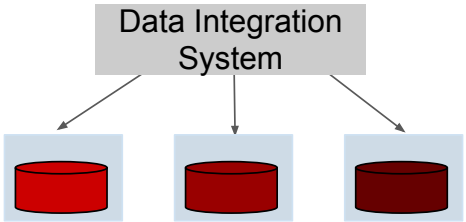
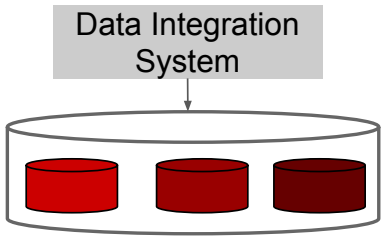


Types of Data Integration Systems (1/2)

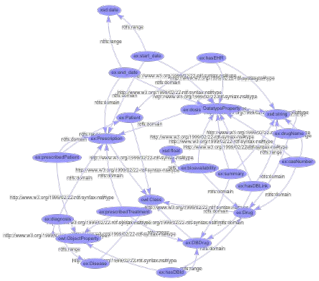


- **Centralized:** All data is stored in a single repository, providing a unified access point, but requiring high control and maintenance of a central system.
- **Distributed:** Data resides in multiple locations, and integration occurs dynamically, allowing scalability but adding complexity to synchronization and access.
- **Homogeneous:** All data sources follow the same schema or model, simplifying integration while limiting flexibility in handling diverse data structures.
- **Heterogeneous:** Data sources have different schemata or formats, requiring mapping and transformation to provide a uniform view, but enabling integration across varied systems.

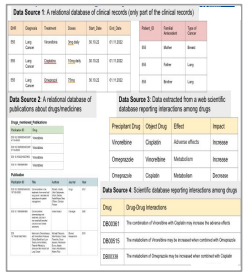
Types of Data Integration Systems (2/2)

Type of Data Integration System	Description
	**Centralized-Homogeneous** Data Integration System. DIS=<O,S,M>, the number of data sources in S is 1.
	**Distributed-Homogeneous** Data Integration System. DIS=<O,S,M>, all the data sources in S are represented using the same data model and schema, i.e., there are no heterogeneity conflicts.
	**Distributed-Heterogeneous** Data Integration System. DIS=<O,S,M>, data sources in S may suffer from heterogeneity conflicts and are distributed in different locations.
	**Centralized-Heterogeneous** Data Integration System. DIS=<O,S,M>, data sources in S may suffer from heterogeneity conflicts, but all are in the same locations.

Data Integration Systems and Knowledge Graphs

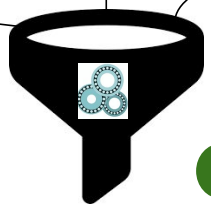


Ontology describes the universe of discourse and provides a unified view of the data sources.



Data sources that provide the data that will be integrated.

Mapping Rules establish the correspondences between the ontology and the data sources.



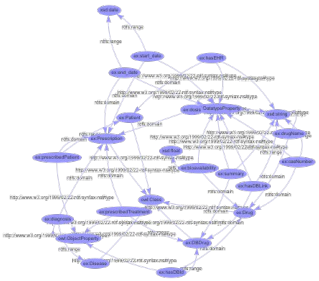
1. The **execution of the mapping rules** on the data sources creates a knowledge graph.

The **knowledge graph** comprises the instances of the classes and properties in the ontology; it also includes the ontology.

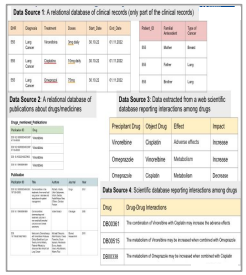
2. **Quality of the data integrated** in the knowledge graph is validated using integrity constraints.

3. **Queries** express graph patterns to be evaluated over the knowledge graph.

Data Integration Systems and Knowledge Graphs

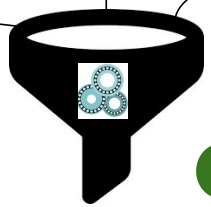


Ontology describes the universe of discourse and provides a unified view of the data sources.



Data sources that provide the data that will be integrated.

Mapping Rules establish the correspondences between the ontology and the data sources.



1. The **execution of the mapping rules** on the data sources creates a knowledge graph.

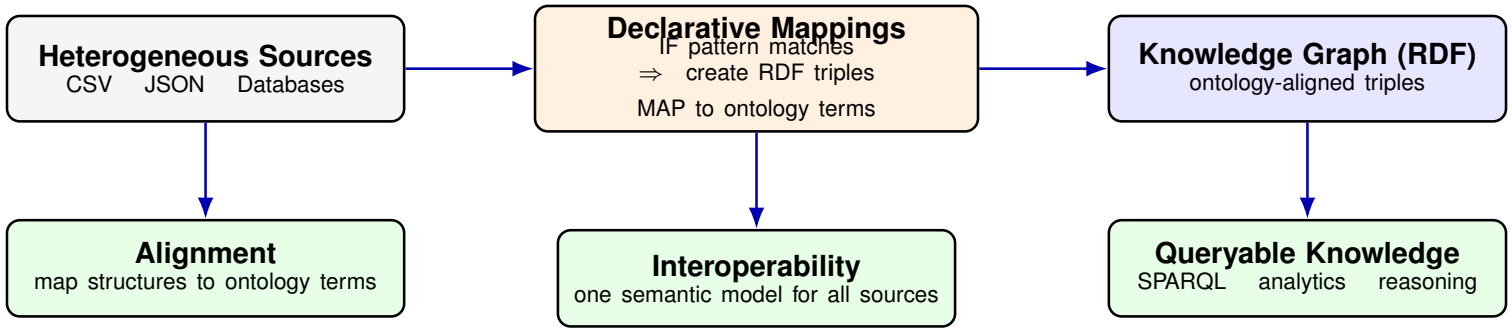
2. **Quality of the data integrated** in the knowledge graph is validated using integrity constraints.

3. **Queries** express graph patterns to be evaluated over the knowledge graph.

The **knowledge graph** comprises the instances of the classes and properties in the ontology; it also includes the ontology.

This pipeline demonstrates how a Data Integration System (DIS) transforms multiple data sources into a unified Knowledge Graph through mapping rules and ontology alignment.

Declarative Mapping Languages



Takeaway: Declarative mappings are **rules (not code)** that transform heterogeneous data into a **uniform, ontology-aligned KG**.

Declarative Mapping Languages

What are Declarative Mapping Languages?

A **declarative mapping language** specifies *rules* (not code) that describe **how data from heterogeneous sources is transformed into a knowledge graph**.

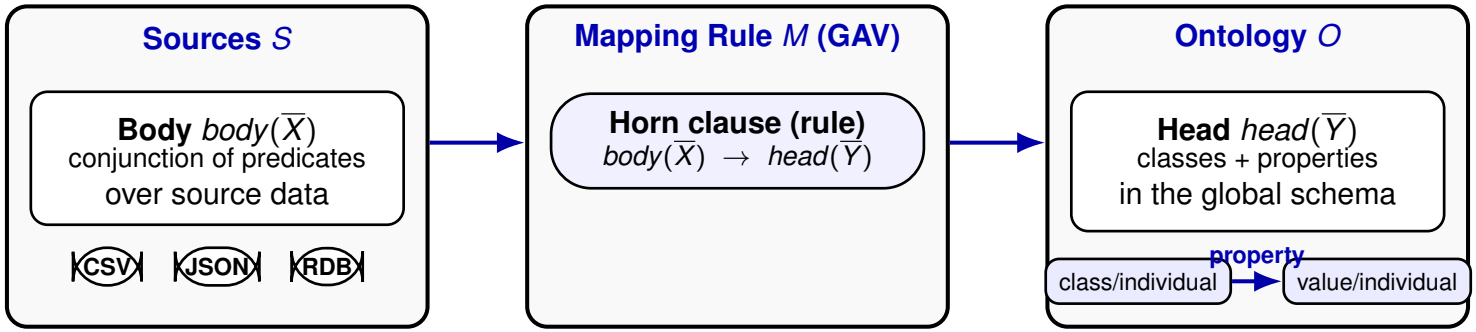
Key Characteristics

- Rule-based specifications
- Independent of execution strategy
- Explicit semantics

Why They Matter

- **Schema alignment:** map source structures to ontologies
- **Interoperability:** integrate diverse data sources
- **Queryability:** produce structured, RDF-based knowledge

Mapping Rules or Assertions (1/2)



Variable condition:
every variable in the head also appears in the body.

GAV intuition
each ontology element is defined
as a view over the sources

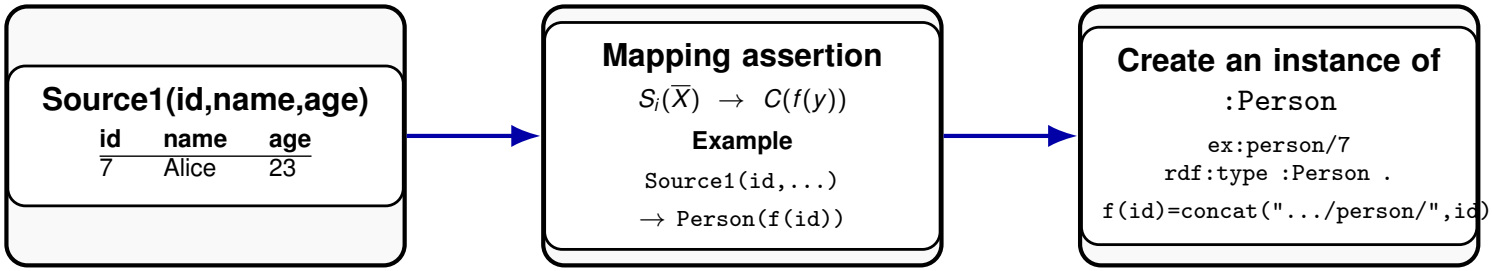
Types of mapping assertions: Concept | Role | Attribute (abstract view of R2RML/RML triple maps)

Mapping Rules or Assertions (2/2)

Definition: Mapping rules in M are formalized as Horn clauses $body(\overline{X})$-:$head(\overline{Y})$

- **Global As View (GAV) approach:**
  - $body(\overline{X})$: Conjunction of predicates over the sources S.
  - $head(\overline{Y})$: Represents classes and properties in the ontology O.
  - Variables in $head(\overline{Y})$ appear in $body(\overline{X})$.
- Abstract representation of triple maps in R2RML or RML.
- Types of mapping assertions:
  - Concept Mapping Assertions
  - Role Mapping Assertions
  - Attribute Mapping Assertions

Concept Mapping Assertions (1/2)



Source S_i **Concept rule** **Ontology instance**

Takeaway: one source row ⇒ one new IRI ⇒ one class instance.

Concept Mapping Assertions (2/2)

Definition:

- Create instances of a class C from ontology O based on S_i.
- Rule format: $S_i(\overline{X})$-:$C(f(y))$
- $f(.)$: Function for generating unique identifiers (e.g., concatenation).

Concept Mapping Assertions (2/2)

Definition:

- Create instances of a class C from ontology O based on S_i.
- Rule format: $S_i(\overline{X})$-:$C(f(y))$
- $f(.)$: Function for generating unique identifiers (e.g., concatenation).

Example: Source1(id, name, age)-:Person(concat("http://example.org/person/",id).

Concept Mapping Assertions (2/2)

Definition:

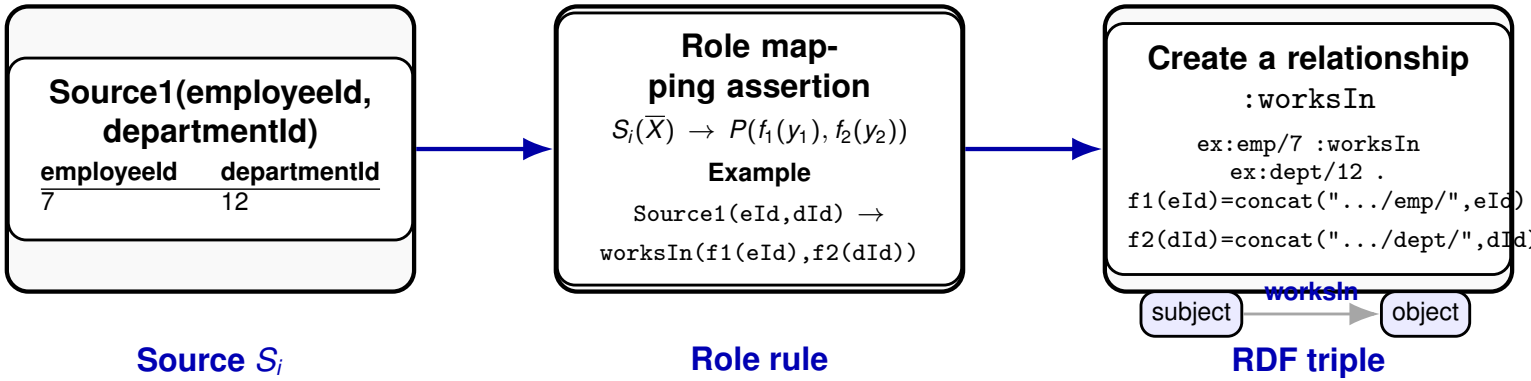
- Create instances of a class C from ontology O based on S_i.
- Rule format: $S_i(\overline{X})$-:$C(f(y))$
- $f(.)$: Function for generating unique identifiers (e.g., concatenation).

Example: Source1(id, name, age)-:Person(concat("http://example.org/person/",id).

Explanation:

- The evaluation of this rule creates instances of the class `Person`.
- Each instance is identified by a unique URI generated from the `id` attribute in the source.

Role Mapping Assertions: Single-Source (1/2)



Takeaway: one source row ⇒ one subject IRI + one object IRI ⇒ one `worksIn` RDF triple.

Role Mapping Assertions: Single-Source (2/2)

Definition:

- Define a role $P(.,.)$ based on a single data source.
- Rule format: $S_i(\overline{X})$ -: $P(f_1(y_1), f_2(y_2))$
- Subject and object URIs are generated using functions f_1 and f_2 applied to attributes y_1 and y_2.

Role Mapping Assertions: Single-Source (2/2)

Definition:

- Define a role $P(.,.)$ based on a single data source.
- Rule format: $S_i(\overline{X})$ -: $P(f_1(y_1), f_2(y_2))$
- Subject and object URIs are generated using functions f_1 and f_2 applied to attributes y_1 and y_2.

Example:

```
Source1(employeeId, departmentId) -:  worksIn({employeeId}, {departmentId}).
```

Role Mapping Assertions: Single-Source (2/2)

Definition:

- Define a role $P(.,.)$ based on a single data source.
- Rule format: $S_i(\overline{X})$ -: $P(f_1(y_1), f_2(y_2))$
- Subject and object URIs are generated using functions f_1 and f_2 applied to attributes y_1 and y_2.

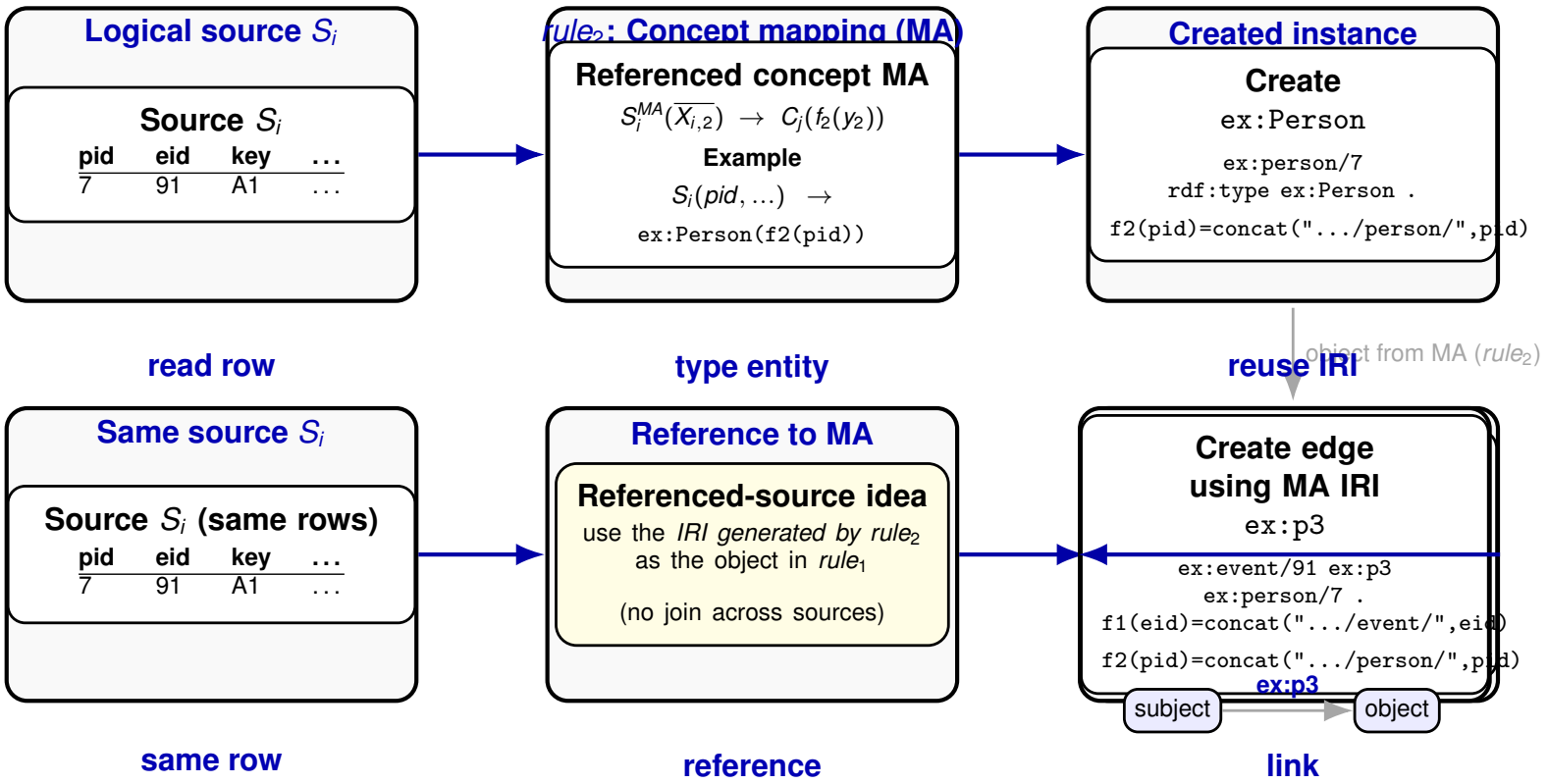
Example:

```
Source1(employeeId, departmentId) -:  worksIn({employeeId}, {departmentId}).
```

Explanation:

- This rule creates relationships of type `worksIn`.
- Both the subject (`employeeId`) and the object (`departmentId`) are generated from a single data source.

Role Mapping Assertions: Referenced-Source



Takeaway: both rules use the **same source**; $rule_2$ creates the **IRI**, and $rule_1$ references it to build **ex:p3**.

Role Mapping Assertions: Referenced-Source (2/2)

Definition:

- Specify the object value of a role $P(.,.)$ based on a referenced concept mapping assertion (`MA`).
- Rule format:
  $rule_1$: $S_i(\overline{X_{i,1}}), S_i^{MA}(\overline{X_{i,2}})$ -: $P(f_1(y_1), f_2(y_2))$ where
  $rule_2$: $S_i^{MA}(\overline{X_{i,2}})$ -: $C_j(f_2(y_2))$

Role Mapping Assertions: Referenced-Source (2/2)

Definition:

- Specify the object value of a role $P(.,.)$ based on a referenced concept mapping assertion (`MA`).
- Rule format:
  $rule_1$: $S_i(\overline{X_{i,1}}), S_i^{MA}(\overline{X_{i,2}})$ -: $P(f_1(y_1), f_2(y_2))$ where
  $rule_2$: $S_i^{MA}(\overline{X_{i,2}})$ -: $C_j(f_2(y_2))$

Example:

- $rule_2$ defines instances of a class, e.g., `ex:Person`.
- $rule_1$ defines a property, e.g., `ex:p3`, linking a person (from $rule_2$) to another entity in the same data source.

Role Mapping Assertions: Referenced-Source (2/2)

Definition:

- Specify the object value of a role $P(.,.)$ based on a referenced concept mapping assertion (`MA`).
- Rule format:
  $rule_1$: $S_i(\overline{X_{i,1}}), S_i^{MA}(\overline{X_{i,2}})$ -: $P(f_1(y_1), f_2(y_2))$ where
  $rule_2$: $S_i^{MA}(\overline{X_{i,2}})$ -: $C_j(f_2(y_2))$

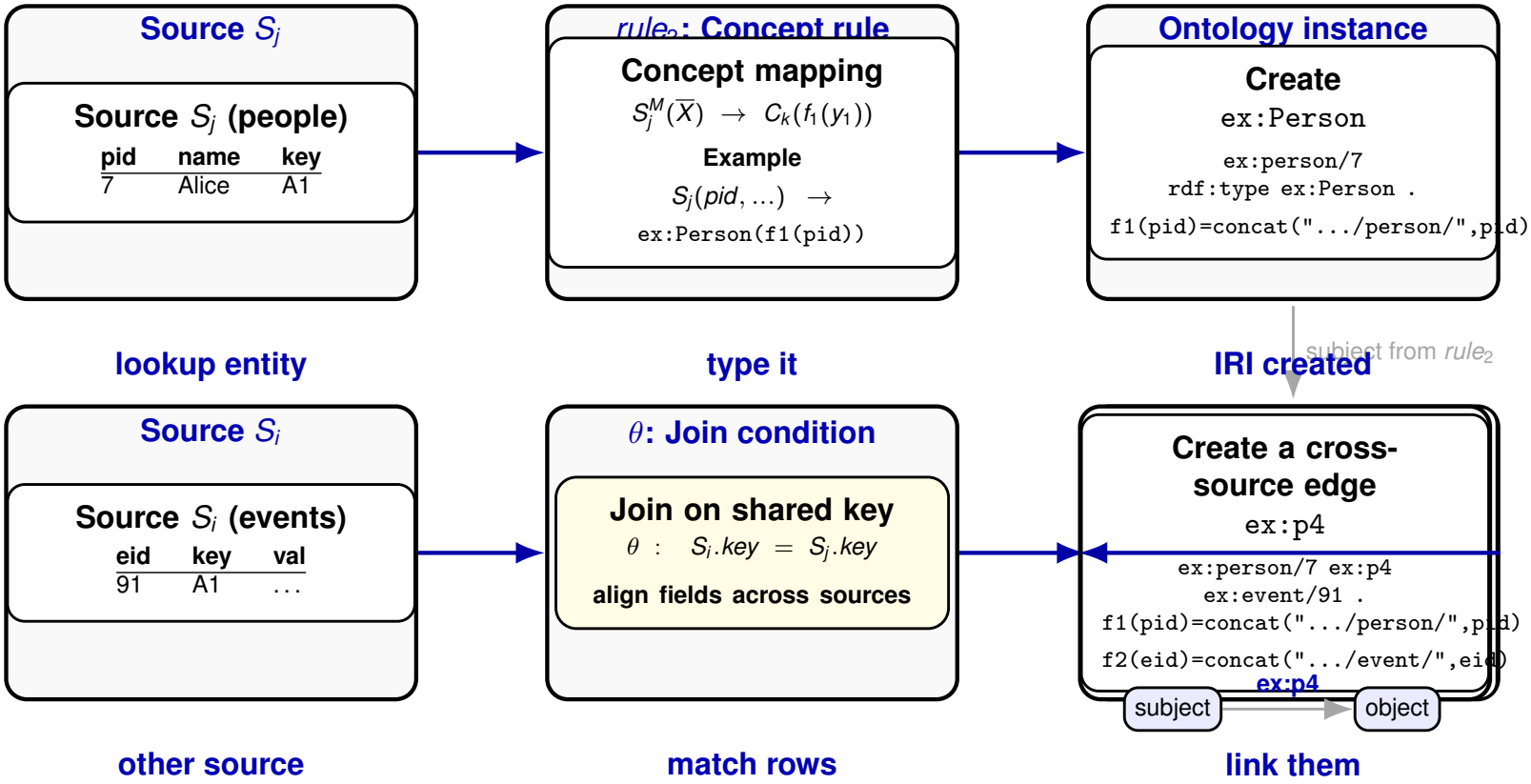
Example:

- $rule_2$ defines instances of a class, e.g., `ex:Person`.
- $rule_1$ defines a property, e.g., `ex:p3`, linking a person (from $rule_2$) to another entity in the same data source.

Explanation:

- The object value of `ex:p3` in $rule_1$ is derived using the subject generated by $rule_2$.
- Both $rule_2$ and $rule_1$ work over the same logical source, ensuring alignment.

Multi-Source Role Mapping Assertions (1/2)



Takeaway: *rule*$_2$ creates the **subject entity**; *rule*$_1$ uses a **join** to link it to an **object from another source**.

Multi-Source Role Mapping Assertions (2/2)

Definition: Multi-source role mapping assertions define roles $P(.,.)$ where the subject and object come from different sources, S_j and S_i, respectively. A join condition θ is required.

$rule_1$: $S_i(\overline{X_{i,1}}), S_j^M(\overline{X_{j,2}}), \theta(\overline{X_{i,1}}, \overline{X_{j,2}})$-:$P(f_1(y_1), f_2(y_2))$, where

$rule_2$: $S_j^M(\overline{X_{j,2}})$-:$C_k(f_1(y_1))$

Multi-Source Role Mapping Assertions (2/2)

Definition: Multi-source role mapping assertions define roles $P(.,.)$ where the subject and object come from different sources, S_j and S_i, respectively. A join condition θ is required.

$rule_1$: $S_i(\overline{X_{i,1}}), S_j^M(\overline{X_{j,2}}), \theta(\overline{X_{i,1}}, \overline{X_{j,2}})$-:$P(f_1(y_1), f_2(y_2))$, where

$rule_2$: $S_j^M(\overline{X_{j,2}})$-:$C_k(f_1(y_1))$

Example:

- $rule_2$ defines instances of a class, e.g., `ex:Person`.
- $rule_1$ defines a property, e.g., `ex:p4`, linking an entity (from $rule_1$) to another in a different data source.

Multi-Source Role Mapping Assertions (2/2)

Definition: Multi-source role mapping assertions define roles $P(.,.)$ where the subject and object come from different sources, S_j and S_i, respectively. A join condition θ is required.

$rule_1$: $S_i(\overline{X_{i,1}}), S_j^M(\overline{X_{j,2}}), \theta(\overline{X_{i,1}}, \overline{X_{j,2}})$-:$P(f_1(y_1), f_2(y_2))$, where

$rule_2$: $S_j^M(\overline{X_{j,2}})$-:$C_k(f_1(y_1))$

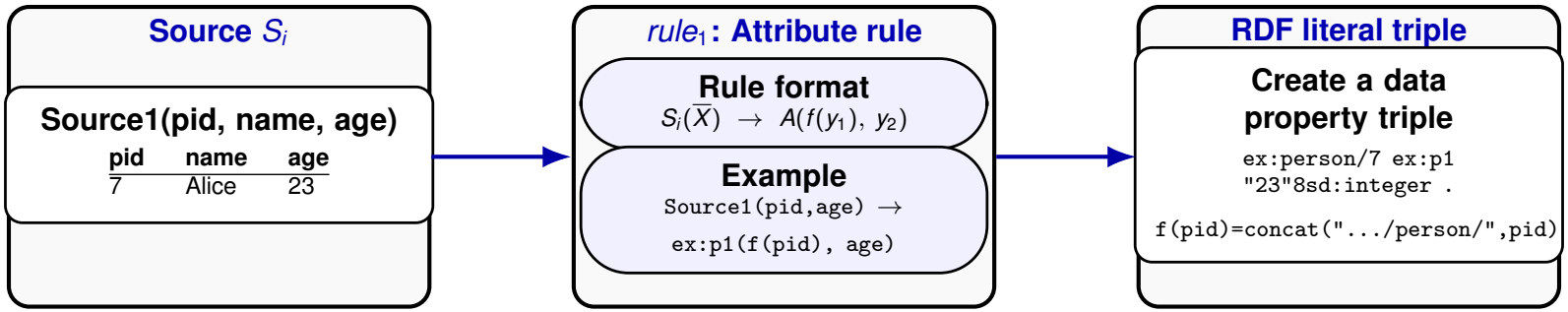
Example:

- $rule_2$ defines instances of a class, e.g., `ex:Person`.
- $rule_1$ defines a property, e.g., `ex:p4`, linking an entity (from $rule_1$) to another in a different data source.

Explanation:

- $rule_1$ uses a join condition to align fields across two sources (S_i and S_j).
- The join ensures that the correct subject and object are linked via `ex:p4`.

Attribute Mapping Assertions (1/2)



Takeaway: one row $\Rightarrow$ create subject IRI with $f(y_1)$ and attach a literal value y_2.

Attribute Mapping Assertions (2/2)

Definition: Attribute mapping assertions define data properties A, where the subject is derived using a function $f(y_1)$, and the object value is a literal obtained from the source.

$rule_1$: $S_i(\overline{X})$-:$A(f(y_1), y_2)$

Attribute Mapping Assertions (2/2)

Definition: Attribute mapping assertions define data properties A, where the subject is derived using a function $f(y_1)$, and the object value is a literal obtained from the source.

$rule_1$: $S_i(\overline{X})$-:$A(f(y_1), y_2)$

Example:

- $rule_1$ defines an attribute, e.g., `ex:p1`, where the subject is generated using $f(y_1)$ from the source, and the object is retrieved as a literal value.

Attribute Mapping Assertions (2/2)

Definition: Attribute mapping assertions define data properties A, where the subject is derived using a function $f(y_1)$, and the object value is a literal obtained from the source.

$rule_1$: $S_i(\overline{X})$-:$A(f(y_1), y_2)$

Example:

- $rule_1$ defines an attribute, e.g., `ex:p1`, where the subject is generated using $f(y_1)$ from the source, and the object is retrieved as a literal value.

Explanation:

- Attribute mapping assertions map source attributes to RDF literals using predefined functions.
- The subject is created using a function applied to the source's attributes, and the object is directly retrieved as a literal.

Example (1/5)

Data Sources S:

Drug

DrugName	Bioavailability	CasNumber	DrugBankID
Vinorelbine	43.000000	71486-22-1	DB00361
Cisplatine	100.000000	15663-27-1	DB00515
Omeprazole	35.000000	73590-58-6	DB00338

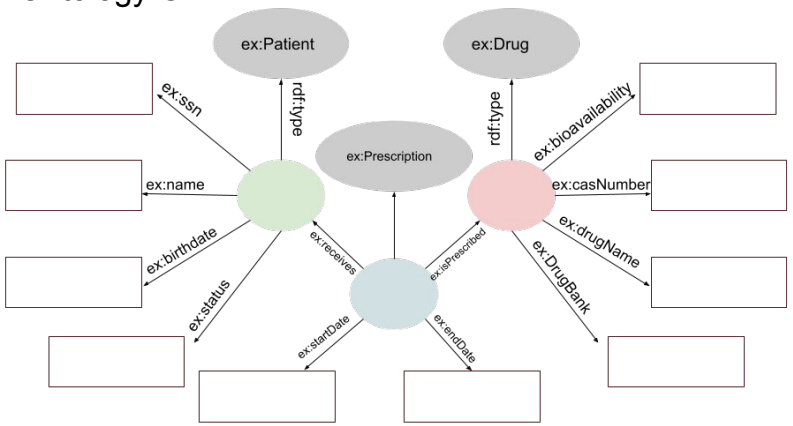
Treatment

DrugID	PatientID	StartDate	EndData
DB00361	551	20.01.2021	31.03.2021
DB00515	551	20.01.2021	31.03.2021
DB00338	551	20.01.2021	31.03.2021

Patient

SSN	Name	Birthdate	Status
551	John Smith	20.12.1978	Alive with disease
552	Peter Lange	19.01.2010	Dead
553	Luis Perez	14.01.1959	Healthy

Ontology O:



Example (2/5)

Data Sources:

- **Drug:** Contains information about drugs, including their name, bioavailability, CAS number, and DrugBank ID.
- **Patient:** Includes patient details such as Social Security Number (SSN), name, birthdate, and health status.
- **Treatment:** Links patients and drugs with start and end dates of treatments.

Ontology:

- `ex:Patient:` Represents patients with attributes like name, birthdate, and status.
- `ex:Drug:` Represents drugs with properties such as name, bioavailability, CAS number, and DrugBank ID.
- `ex:Prescription:` Represents the treatment relationship between patients and drugs, including the start and end dates.

Concept Mapping Assertions (3/5)

```
Patient(SSN, Name, Birthdate, Status) -:
ex:Patient(concat("www.example.org/",SSN)).

Drug(DrugBankID, DrugName, Bioavailability, CasNumber) -:
ex:Drug(concat("www.example.org/",DrugBankID)).

Treatment(DrugID, PatientID, StartDate, EndDate)-:
ex:Prescription(concat("www.example.org/",concat(DrugID, PatientID))).
```

Concept Mapping Assertions (3/5)

```
Patient(SSN, Name, Birthdate, Status) -:
ex:Patient(concat("www.example.org/",SSN)).

Drug(DrugBankID, DrugName, Bioavailability, CasNumber) -:
ex:Drug(concat("www.example.org/",DrugBankID)).

Treatment(DrugID, PatientID, StartDate, EndDate)-:
ex:Prescription(concat("www.example.org/",concat(DrugID, PatientID))).
```

- Patient data from the ‘Patient‘ source are mapped to the ‘ex:Patient‘ class.
- Treatment data from the ‘Treatment‘ source are mapped to the ‘ex:Prescription‘ class.
- Drug data from the ‘Drug‘ source are mapped to the ‘ex:Drug‘ class.

Attribute Mapping Assertions (4/5)

```
# Patient Attributes
Patient(SSN,Name) -: ex:name(concat("www.example.org/",SSN), Name).
Patient(SSN,Birthdate) -: ex:birthdate(concat("www.example.org/",SSN),Birthdate).
Patient(SSN,Status) -: ex:status(concat("www.example.org/",SSN),Status).
Patient(SSN,Status) -: ex:ssn(concat("www.example.org/",SSN),SSN).
# Drug Attributes
Drug(DrugBankID,DrugName) -: ex:name(concat("www.example.org/",DrugBankID), DrugName).
Drug(DrugBankID, Bioavailability) -:
ex:bioavailability(concat("www.example.org/",DrugBankID), Bioavailability).
Drug(DrugBankID,CasNumber) -:
ex:casNumber(concat("www.example.org/",DrugBankID),CasNumber).
Drug(DrugBankID,CasNumber) -:
ex:DrugBank(concat("www.example.org/",DrugBankID),CasNumber).
# Prescription Attributes
Prescription(DrugID, PatientID, StartDate) -:
ex:startDate(concat("www.example.org/",concat(DrugID,PatientID)), StartDate).
Prescription(DrugID, EndDate) -:
ex:endDate(concat("www.example.org/",concat(DrugID,PatientID)), EndDate).
```

Attribute Mapping Assertions (4/5)

```
# Patient Attributes
Patient(SSN,Name) -: ex:name(concat("www.example.org/",SSN), Name).
Patient(SSN,Birthdate) -: ex:birthdate(concat("www.example.org/",SSN),Birthdate).
Patient(SSN,Status) -: ex:status(concat("www.example.org/",SSN),Status).
Patient(SSN,Status) -: ex:ssn(concat("www.example.org/",SSN),SSN).
# Drug Attributes
Drug(DrugBankID,DrugName) -: ex:name(concat("www.example.org/",DrugBankID), DrugName).
Drug(DrugBankID, Bioavailability) -:
ex:bioavailability(concat("www.example.org/",DrugBankID), Bioavailability).
Drug(DrugBankID,CasNumber) -:
ex:casNumber(concat("www.example.org/",DrugBankID),CasNumber).
Drug(DrugBankID,CasNumber) -:
ex:DrugBank(concat("www.example.org/",DrugBankID),CasNumber).
# Prescription Attributes
Prescription(DrugID, PatientID, StartDate) -:
ex:startDate(concat("www.example.org/",concat(DrugID,PatientID)), StartDate).
Prescription(DrugID, EndDate) -:
ex:endDate(concat("www.example.org/",concat(DrugID,PatientID)), EndDate).
```

Explanation:

- Maps attributes in data sources to properties of instances in the ontology.
- Ensures consistency by aligning literal values with the appropriate subjects.

Multi-Source Role Mapping Assertions (5/5)

```
# Reference Rule for Patient
rule_1: Patient(SSN) -: ex:Patient(concat("www.example.org/", SSN)).
# Reference Rule for Drug
rule_2: Drug(DrugBankID) -: ex:Drug(concat("www.example.org/", DrugBankID)).
# Patient to Prescription Relationship (ex:prescribes)
rule_3: Treatment(PatientID, DrugID), rule_1(PatientID) -:
ex:prescribes(concat("www.example.org/", PatientID),
              concat("www.example.org/", concat(PatientID, DrugID))).
# Prescription to Drug Relationship (ex:refersTo)
rule_4: Treatment(PatientID, DrugID), rule_2(DrugID) -:
ex:refersTo(concat("www.example.org/", concat(PatientID, DrugID)),
            concat("www.example.org/", DrugID)).
```

Multi-Source Role Mapping Assertions (5/5)

```
# Reference Rule for Patient
rule_1: Patient(SSN) -: ex:Patient(concat("www.example.org/", SSN)).
# Reference Rule for Drug
rule_2: Drug(DrugBankID) -: ex:Drug(concat("www.example.org/", DrugBankID)).
# Patient to Prescription Relationship (ex:prescribes)
rule_3: Treatment(PatientID, DrugID), rule_1(PatientID) -:
ex:prescribes(concat("www.example.org/", PatientID),
              concat("www.example.org/", concat(PatientID, DrugID))).
# Prescription to Drug Relationship (ex:refersTo)
rule_4: Treatment(PatientID, DrugID), rule_2(DrugID) -:
ex:refersTo(concat("www.example.org/", concat(PatientID, DrugID)),
            concat("www.example.org/", DrugID)).
```

Explanation:

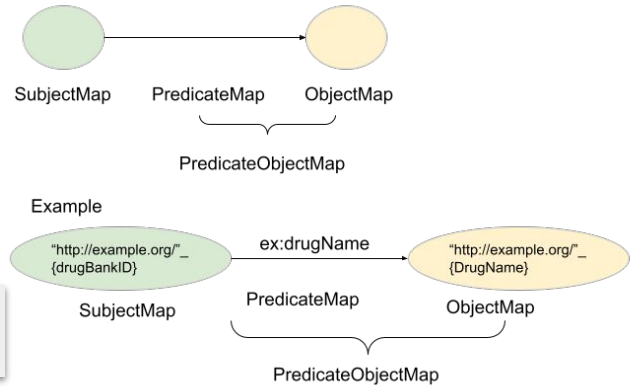
- **rule_1** defines instances of `ex:Patient`.
- **rule_2** defines instances of `ex:Drug`.
- **rule_3** references **rule_1** to define the relationship `ex:prescribes`, linking patients to prescriptions.
- **rule_4** references **rule_2** and includes `Treatment(PatientID, DrugID)` to define the relationship `ex:refersTo`, linking prescriptions to drugs.
- The join conditions are modeled explicitly, ensuring correct alignment of subjects and objects across sources.

RDF Triple Centric Mapping Language (1/3)

Drug

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338

Mapping Rule or PredicateObjectMap



Key Points:

- A Mapping Rule, often termed a `PredicateObjectMap`, is central to RDF triple-centric mapping languages.
- It specifies how predicates and objects in RDF triples are derived from data sources.
- The `PredicateObjectMap` consists of two primary components:
  - `rr:predicateMap`: Defines the RDF predicate for the mapping.
  - `rr:objectMap`: Specifies the value of the RDF object, derived from data in the source.
- These mappings transform data into RDF triples aligning them with an ontology.

RDF Triple Centric Mapping Language (2/3)

Drug

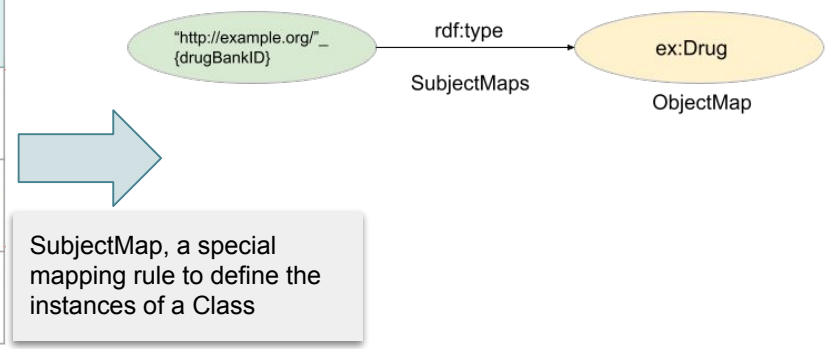
DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338

SubjectMap, a special mapping rule to define the instances of a Class

RDF Triple Centric Mapping Language (2/3)

Drug

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338



Key Points:

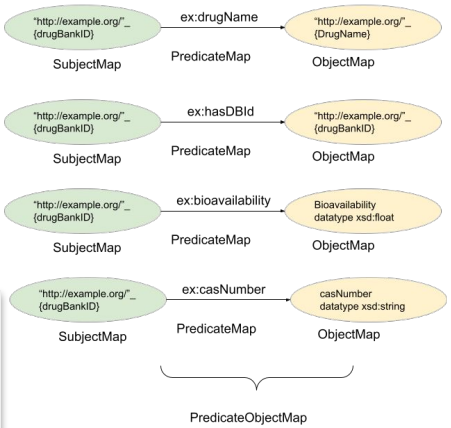
- **SubjectMap:** A specialized mapping rule used to define the instances of a class in the ontology.
- **Instance Generation:** Each subject in the RDF graph is associated with a unique URI generated based on the data source.
- **Uniform Resource Identification:** Subject URIs are created by combining attributes from the data source using predefined templates or functions.

RDF Triple Centric Mapping Language (3/3)

Drug

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338

Various PredicateObjectMap Mapping Rules

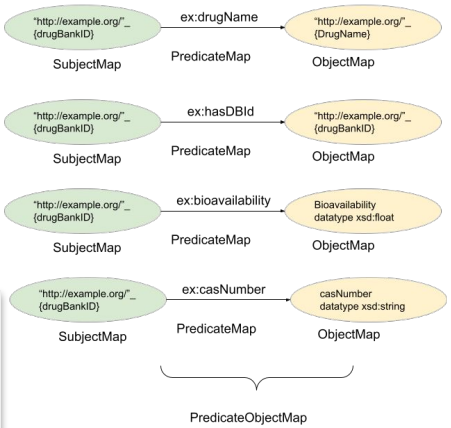


RDF Triple Centric Mapping Language (3/3)

Drug

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338

Various PredicateObjectMap Mapping Rules



Explanation:

- Each `PredicateObjectMap` associates a specific predicate (property) with its corresponding object (value).
- Supports complex mappings by referencing other mapping assertions.

Mapping Languages

Given a **data integration system** $DIS = \langle O, S, M \rangle$:

What is a mapping language?

- A **mapping language** is a **declarative programming language** to express how the data from the data sources in S is used to populate the concepts (i.e., classes and properties) in O.

Why are mapping languages needed?

1. To define the **identifiers** (i.e., IRIs) of the entities of the **classes in** O based on the **attributes of sources in** S.
2. To define the **properties** (data type and object properties) in O in terms of **attributes of the data sources in** S.

Mapping Languages

Given a **data integration system** $DIS = \langle O, S, M \rangle$:

Why is it important that a mapping language is declarative?

1. A **declarative mapping language** provides a high-level description of the definitions of the concepts of the ontology O in the data sources in S.
2. A **declarative mapping language** expresses **"WHAT"** are the conditions to be met for the attributes of a data source in S to define the concepts (i.e., classes or properties) in O.
3. A **declarative mapping language** hides implementation details of **HOW** the data of a data source is used to create instances of the concepts (i.e., classes or properties) in O.

Why not procedural language for mappings? (1/2)

Given a **data integration system** $DIS = \langle O, S, M \rangle$:

Why do not simply use a procedural language (e.g., Python or Java) to define mappings in M?

- Procedural languages could be used to implement the mappings between the concepts in O and data sources in S.
- However, these implementations could be challenging to:
  - **Maintain, reuse, or trace.**

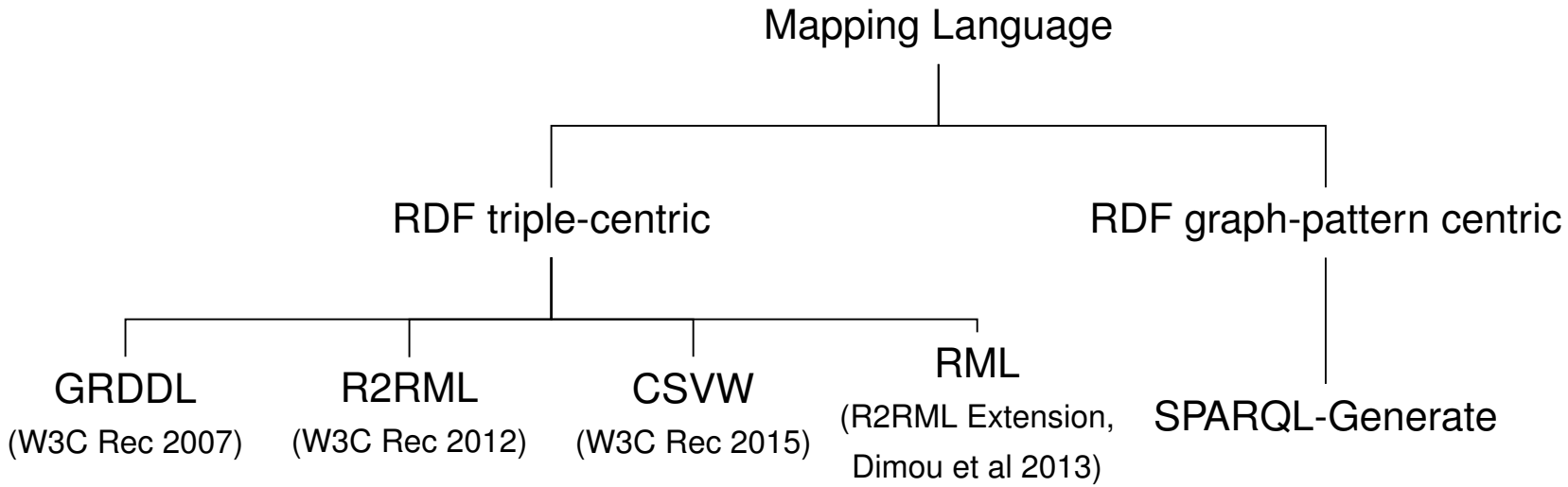
Why not procedural language for mappings? (2/2)

Why are we learning R2RML and RML?

- **R2RML** is the W3C standard to represent mapping rules between concepts in ontology *O* and relational data sources in *S*.
- **RML** is an extension of R2RML to represent mapping rules between concepts in *O* and data sources in *S* in the formats CSV, XML, JSON, or relational databases.
- R2RML and RML are **declarative mapping languages** expressed in RDF.

Mapping Languages in RDF (1/2)

Declarative Mapping Languages

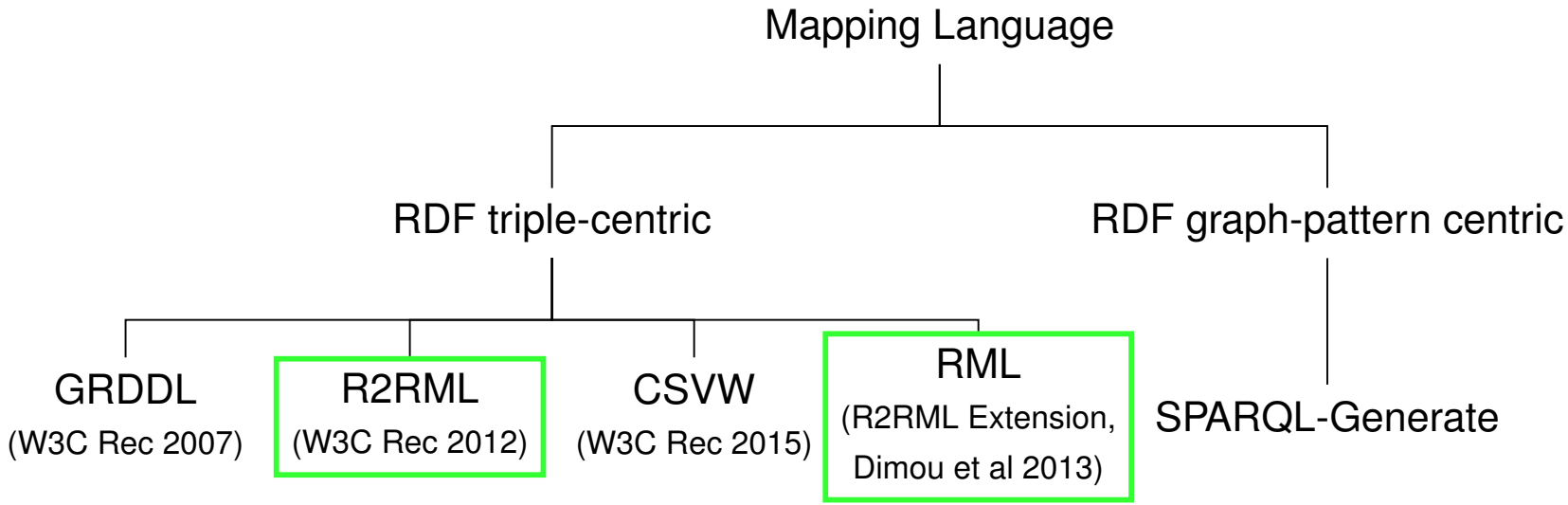


Declarative Mapping Languages in RDF

Declarative mapping languages in RDF define rules for translating data from diverse formats (e.g., relational, CSV, XML, JSON) into RDF triples. These languages enable the creation of interoperable Knowledge Graphs by aligning heterogeneous data with a unified ontology.

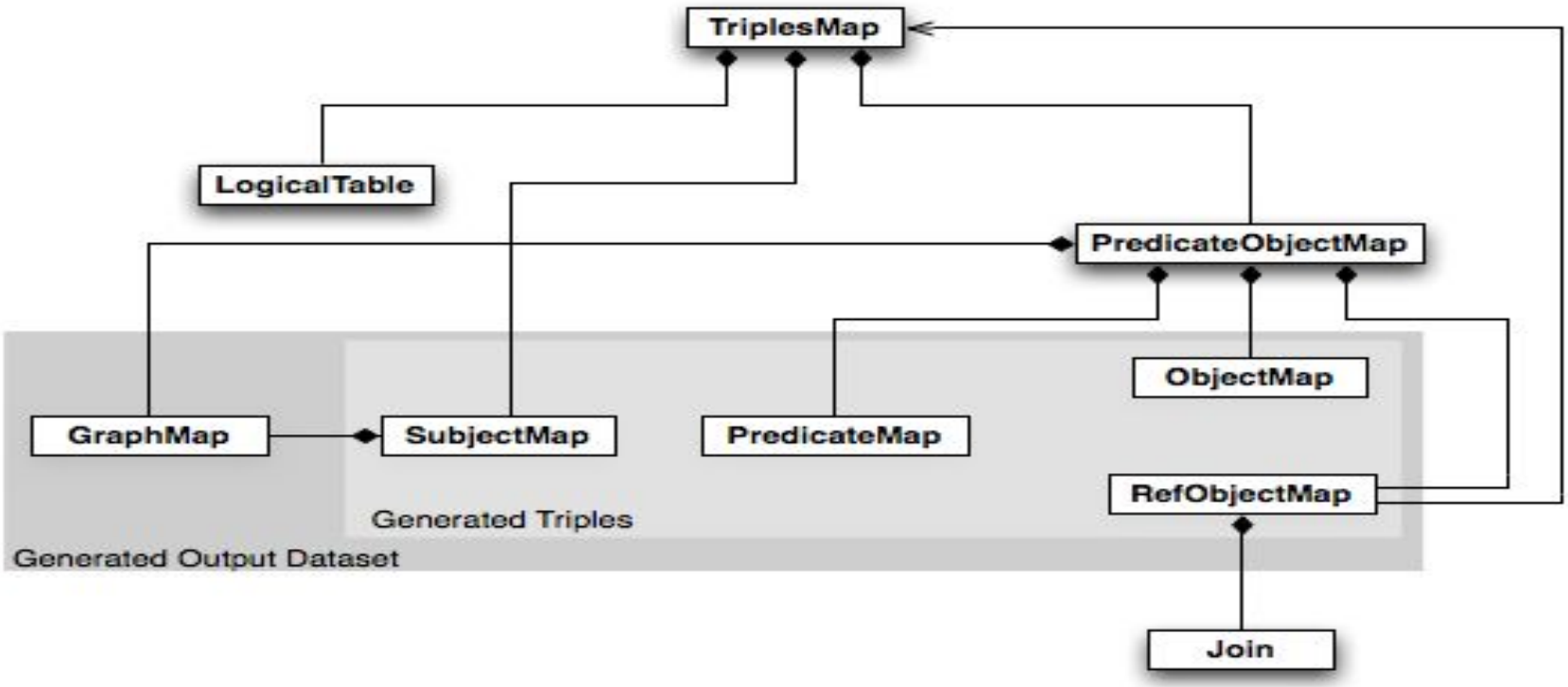
Mapping Languages in RDF (2/2)

Declarative Mapping Languages in RDF



R2RML

- Mapping Language to specify the mapping rules from relational databases into RDF. R2RML is expressed in RDF. W3C Recommendation Language - 2012.
  `https://www.w3.org/TR/r2rml/`



R2RML vs Mapping Assertions

Mapping Component	R2RML Equivalent	Explanation
Concept Mapping Assertions	`rr:subjectMap`	Defines the subject of RDF triples, maps data from relational tables to instances of classes in the ontology.
Role Mapping Assertions (Single Source)	`rr:predicateObjectMap` with `rr:objectMap`	Maps relational attributes to object properties where subject and object are derived from the same data source.
Role Mapping Assertions (Referenced Source)	`rr:RefObjectMap` with `rr:parentTriplesMap`	Links object properties to subjects defined in another triples map, often requiring join conditions.
Role Mapping Assertions (Multi-Source)	`rr:RefObjectMap` with `rr:joinCondition`	Links data across multiple sources by specifying join conditions to define subject-object relationships.
Attribute Mapping Assertions	`rr:predicateObjectMap` with `rr:column`	Maps relational attributes to datatype properties in RDF, ensuring consistency for literal values.
Source Definition in the body of an assertion	`rr:logicalTable`	Defines table or SQL query used to extract data for triples generation.

R2RML

- Mapping rules **(TriplesMaps)** from relational tables into RDF **(LogicalTable)**
  - a base table
  - a valid SQL query
- A **SubjectMap** specifies the subject of generated RDF triples
- A **PredicateObjectMap** assigns a predicate and object to a subject described using a **SubjectMap**
  - **PredicateMap** indicates the predicate of the generated RDF triple
  - **ObjectMap** defines the object of the generated RDF triple

R2RML - Definitions

- Each triples map consists of one logical table, one subject map, and zero or more predicate-object maps.

LogicalTable:

A logical table (**rr:logicalTable**) states the relational table from where data is collected; it can be defined as follows:

- Table (**rr:tableName**) - name of the table;
- SQL query (**rr:sqlQuery**) - SQL query whose execution corresponds to the data source. The version of SQL can be specified with **rr:sqlVersion**, e.g., **rr:sqlVersion rr:SQL2008**.

SubjectMap:

- A subject map (**rr:subjectMap**) - defines the subject of the generated RDF triples.
- The predicate (**rr:class**) is utilized to specify the type of the subject
- **rr:template** indicates the template that will be followed to create the identifier of the subject.

Example - Simple Ontology & Data Source

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
@prefix owl: <http://www.w3.org/2002/07/owl#>.

ex:Drug a owl:Class.
ex:DBDrug a owl:Class.

ex:drugName a owl:DatatypeProperty;
            rdfs:domain ex:Drug;
            rdfs:range xsd:string.

ex:hasDBId a owl:ObjectProperty;
           rdfs:domain ex:Drug;
           rdfs:range ex:DBDrug.

ex:bioavailability a owl:DatatypeProperty;
                   rdfs:domain ex:Drug;
                   rdfs:range xsd:float.

ex:casNumber a owl:DatatypeProperty;
             rdfs:domain ex:Drug;
             rdfs:range xsd:string.

ex:hasDBLink a owl:DatatypeProperty;
             rdfs:domain ex:DBDrug;
             rdfs:range xsd:string.

ex:summary a owl:DatatypeProperty;
           rdfs:domain ex:DBDrug;
           rdfs:range xsd:string.
```

Drug

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338

Example - Simple Ontology & Data Source

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
@prefix owl: <http://www.w3.org/2002/07/owl#>.

ex:Drug a owl:Class.
ex:DBDrug a owl:Class.

ex:drugName a owl:DatatypeProperty;
            rdfs:domain ex:Drug;
            rdfs:range xsd:string.

ex:hasDBId a owl:ObjectProperty;
           rdfs:domain ex:Drug;
           rdfs:range ex:DBDrug.

ex:bioavailability a owl:DatatypeProperty;
                   rdfs:domain ex:Drug;
                   rdfs:range xsd:float.

ex:casNumber a owl:DatatypeProperty;
             rdfs:domain ex:Drug;
             rdfs:range xsd:string.

ex:hasDBLink a owl:DatatypeProperty;
             rdfs:domain ex:DBDrug;
             rdfs:range xsd:string.

ex:summary a owl:DatatypeProperty;
           rdfs:domain ex:DBDrug;
           rdfs:range xsd:string.
```

Drug

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338

Logical table is a basic table

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
<TriplesMapDrug>
    rr:logicalTable [ rr:tableName "Drug"];
 rr:subjectMap [rr:template "http://example.org/Drug/{drugBankID}";
                  rr:class ex:Drug].
```

Example - Simple Ontology & Data Source

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
@prefix owl: <http://www.w3.org/2002/07/owl#>.

ex:Drug a owl:Class.
ex:DBDrug a owl:Class.

ex:drugName a owl:DatatypeProperty;
            rdfs:domain ex:Drug;
            rdfs:range xsd:string.

ex:hasDBId a owl:ObjectProperty;
           rdfs:domain ex:Drug;
           rdfs:range ex:DBDrug.

ex:bioavailability a owl:DatatypeProperty;
                   rdfs:domain ex:Drug;
                   rdfs:range xsd:float.

ex:casNumber a owl:DatatypeProperty;
             rdfs:domain ex:Drug;
             rdfs:range xsd:string.

ex:hasDBLink a owl:DatatypeProperty;
             rdfs:domain ex:DBDrug;
             rdfs:range xsd:string.

ex:summary a owl:DatatypeProperty;
           rdfs:domain ex:DBDrug;
           rdfs:range xsd:string.
```

Drug

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	https://go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	https://go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	https://go.drugbank.com/drugs/DB00338

Logical table is a basic table

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
<TriplesMapDrug>
    rr:logicalTable [ rr:tableName "Drug"];
 rr:subjectMap [rr:template "http://example.org/Drug/{drugBankID}";
                rr:class ex:Drug].
```

Alternatively, a logical table defined as a SQL query

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
<TriplesMapDrug>
    rr:logicalTable [ rr:sqlQuery
    """SELECT DISTINCT drugBankID, DrugName, Bioavailability,
                                   casNumber FROM Drug """];
 rr:subjectMap [rr:template "http://example.org/Drug/{drugBankID}";
                rr:class ex:Drug].
```

R2RML Triple Map: Concept Mapping Assertion

```
rr:logicalTable [ rr:tableName "Treatment" ];
rr:subjectMap [
    rr:template "http://example.com/Prescription/{DrugID}_{PatientID}";
    rr:class ex:Prescription;
];
```

Explanation:

- `rr:logicalTable`: Specifies the data source (e.g., table `Treatment`), forming the **body()** of the mapping assertion.
- `rr:subjectMap`: Defines the subject of the RDF triples.
  - `rr:template`: Constructs a unique URI for instances in the class, e.g., combining `DrugID` and `PatientID`, representing $f(y)$, specifically, the concat(.,.) function.
  - `rr:class`: Indicates the RDF class of the subject, e.g., `ex:Prescription`, corresponding to $C(.)$ in the mapping assertion.

R2RML - More Definitions

Predicate-Object Map (`rr:predicateObjectMap`)

- Combines:
  - **Predicate Maps** (`rr:predicate`): Expressing the predicate of the RDF triple.
  - **Object Maps** (`rr:objectMap`): Expressing the object of the RDF triple.
- For object properties:
  - Use `rr:template` to specify the template for creating the identifier of the object.
- For data type properties:
  - Use `rr:column` to specify the column name that provides the value.
- Both column names and template URLs must be enclosed in double quotes (`" "`).

Example: Drug Mapping (1/2)

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
 # Concept Mapping Assertions (SubjectMap)
<TriplesMapDrug>
    rr:logicalTable [ rr:tableName "Drug"];
    rr:subjectMap [rr:template "http://example.org/Drug/{drugBankID}";
                   rr:class ex:Drug];
 # Role Mapping Assertions (ObjectMap)
    rr:predicateObjectMap [
        rr:predicate ex:drugName;
        rr:objectMap [
            rr:template "http://example.org/Drug/{DrugName}"; ]];
    rr:predicateObjectMap [
        rr:predicate ex:hasDBId;
        rr:objectMap [
            rr:template "http://example.org/Drug/{drugBankID}"; ]];
    rr:predicateObjectMap [
        rr:predicate ex:bioavailability;
        rr:objectMap [
            rr:column "Bioavailability"; rr:datatype xsd:float]];
    rr:predicateObjectMap [
        rr:predicate ex:casNumber;
        rr:objectMap [
            rr:column "casNumber"; rr:datatype xsd:string]].
```

Example: Drug Mapping (2/2)

Explanation:

- The **SubjectMap** defines a Concept Mapping Assertion, associating the table "Drug" with the ontology class `ex:Drug` using a URI template.
- The **ObjectMaps** define Role Mapping Assertions, mapping predicates (`ex:drugName, ex:hasDBId, etc.`) to object values using templates or column references.
- Each **predicateObjectMap** links a predicate (property) to its corresponding object values.

Referencing Object Maps in R2RML

Object maps can also be defined using **referencing object maps**.

- A referencing object map states that the subject of another triples map corresponds to the value of the object of the defined RDF triple. (Multi-Source Role Mapping Assertion)
- Defining a referencing object map may require a **join** between the logical tables of the two related triple maps.

Referencing Object Maps in R2RML

Object maps can also be defined using **referencing object maps**.

- A referencing object map states that the subject of another triples map corresponds to the value of the object of the defined RDF triple. (Multi-Source Role Mapping Assertion)
- Defining a referencing object map may require a **join** between the logical tables of the two related triple maps.

Representation:

- A referencing object map is represented as follows:
  - Exactly one `rr:parentTriplesMap` property whose value must be a triples map.
  - It is called the **parent triple map** of the referencing object map.

Referencing Object Maps in R2RML

Object maps can also be defined using **referencing object maps**.

- A referencing object map states that the subject of another triples map corresponds to the value of the object of the defined RDF triple. (Multi-Source Role Mapping Assertion)
- Defining a referencing object map may require a **join** between the logical tables of the two related triple maps.

Representation:

- A referencing object map is represented as follows:
  - Exactly one `rr:parentTriplesMap` property whose value must be a triples map.
  - It is called the **parent triple map** of the referencing object map.

Example:

- Referencing object maps are useful for defining object properties that link entities across different logical tables.
- Mappings ensure aligns the subject of the parent triple map and the object of the referencing triple map.

R2RML versus Role Mapping Assertion

```
<TriplesMap3>
rr:logicalTable [ rr:tableName "Treatment" ];
rr:subjectMap [
    rr:template "http://example.com/Prescription/{DrugID}_{PatientID}";
    rr:class ex:Prescription;];
rr:predicateObjectMap [
    rr:predicate ex:receives;
    rr:objectMap [
        rr:template "http://example.com/Patient/{PatientID}";
        rr:termType rr:IRI;];];
```

- `rr:logicalTable`: Specifies data sources (e.g., table `Treatment`), forming the **body()** of the mapping assertion.
- `rr:subjectMap`: Defines subject of RDF triples.
  - `rr:template`: Constructs a unique URI for instances in the class `ex:Prescription`, representing $f_1(y)$.
- `rr:predicateObjectMap`: Represents the role (property) and its object.
  - `rr:predicate`: Specifies RDF properties, e.g., `ex:receives`, corresponding to $P(.,.)$.
  - `rr:objectMap`: Defines the object of the RDF triples.
    - `rr:template`: Constructs a URI for the object, e.g., based on `PatientID`, representing $f_2(y)$.
    - `rr:termType`: Declares the object as an IRI.

Example for Parent Triples Maps

Drug Relational Table

DrugName	Bioavailability	casNumber	drugBankID	DBLink
Vinorelbine	43.00	71486-22-1	DB00361	go.drugbank.com/drugs/DB00361
Cisplatin	100.00	15663-27-1	DB00515	go.drugbank.com/drugs/DB00515
Omeprazole	35.00	73590-58-6	DB00338	go.drugbank.com/drugs/DB00338

Example of Referenced Mapping Assertion

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

<TriplesMapDBDrug>
    rr:logicalTable [ rr:sqlQuery """SELECT DISTINCT drugBankID, DBLink FROM Drug"""];
    rr:subjectMap [ rr:template "http://example.org/DBDrug/{drugBankID}";
                    rr:class ex:DBDrug ];

    rr:predicateObjectMap [
        rr:predicate ex:hasDBLink;
        rr:objectMap [
            rr:column "DBLink";
            rr:termType rr:IRI ]].
```

Example of Referenced Mapping Assertion

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

<TriplesMapDBDrug>
    rr:logicalTable [ rr:sqlQuery """SELECT DISTINCT drugBankID, DBLink FROM Drug"""];
    rr:subjectMap [ rr:template "http://example.org/DBDrug/{drugBankID}";
                    rr:class ex:DBDrug ];

    rr:predicateObjectMap [
        rr:predicate ex:hasDBLink;
        rr:objectMap [
            rr:column "DBLink";
            rr:termType rr:IRI ]].
```

Explanation:

- The subject map `rr:subjectMap` defines instances of `ex:DBDrug` identified by the `drugBankID`.
- The object map `rr:objectMap` specifies the `DBLink` column as the object value for the predicate `ex:hasDBLink`.
- `rr:termType rr:IRI` indicates that the object is an IRI.
- The logical table uses an SQL query to retrieve unique `drugBankID` and `DBLink` values.

Example of Referenced Object Map (1/2)

```
<TriplesMapDrug>
    rr:logicalTable [ rr:sqlQuery """SELECT DISTINCT drugBankID, DrugName,
                                     Bioavailability, casNumber FROM Drug """];
    rr:subjectMap [ rr:template "http://example.org/Drug/{drugBankID}";
                    rr:class ex:Drug ];

    rr:predicateObjectMap [
        rr:predicate ex:drugName;
        rr:objectMap [
            rr:template "http://example.org/Drug/{DrugName}"; ]];
    rr:predicateObjectMap [
        rr:predicate ex:bioavailability;
        rr:objectMap [
            rr:column "Bioavailability"; rr:datatype xsd:float]];
    rr:predicateObjectMap [
        rr:predicate ex:casNumber;
        rr:objectMap [
            rr:column "casNumber"; rr:datatype xsd:string]];
    rr:predicateObjectMap [
        rr:predicate ex:hasDBId;
        rr:objectMap [
            rr:parentTriplesMap <TriplesMapDBDrug>]].
```

Example of Referenced Object Map (2/2)

Explanation:

- The subject map `rr:subjectMap` defines instances of `ex:Drug` identified by the `drugBankID`.
- The `rr:predicateObjectMap` with `ex:hasDBId` references the subject of another triples map (`<TriplesMapDBDrug>`).
- This referencing object map establishes a relationship between `Drug` and `DBDrug`.
- The `rr:parentTriplesMap` property links this map to `<TriplesMapDBDrug>`.

R2RML and Referencing Object Map with Join

```
<TriplesMap3>
rr:logicalTable [ rr:tableName "Treatment" ];
rr:subjectMap [
    rr:template "http://example.com/Prescription/{DrugID}_{PatientID}";
    rr:class ex:Prescription;];
rr:predicateObjectMap [
    rr:predicate ex:receives;
    rr:objectMap [
        rr:parentTriplesMap <TriplesMap2>;
        rr:joinCondition [
            rr:child "PatientID";
            rr:parent "SSN";];];];
<TriplesMap2>
rr:logicalTable [ rr:tableName "Patient" ];
rr:subjectMap [
    rr:template "http://example.com/Patient/{SSN}";
    rr:class ex:Patient;];
```

- `rr:logicalTable`: Defines tables (`Treatment` for `TriplesMap3` and `Patient` for `TriplesMap2`).
- `rr:subjectMap`: Specifies unique URIs for the subject of `ex:Prescription` and `ex:Patient`, representing $f_1(y)$ and $f_2(y)$, respectively.
- `rr:predicateObjectMap`: Establishes property `ex:receives` between prescription and patient.
  - `rr:parentTriplesMap`: References the subject of `TriplesMap2`.
  - `rr:joinCondition`: Aligns `PatientID` in `Treatment` with `SSN` in `Patient`.

Example- Parent Triples Maps with Join (1/4)

Drug

drugBankID	DrugName	Bioavailability	casNumber
DB00361	Vinorelbine	43.00	71486-22-1
DB00515	Cisplatin	100.00	15663-27-1
DB00338	Omeprazole	35.00	73590-58-6

DBDrug

ID	DBLink	DBSummary
DB00361	https://go.drugbank.com/drugs/DB00361	Vinca alkaloid used in the treatment of metastatic non-small cell lung carcinoma (NSLC) and in conjunction with other drugs in locally advanced NSCLC.
DB00515	https://go.drugbank.com/drugs/DB00515	Platinum based chemotherapy agent used to treat various sarcomas, carcinomas, lymphomas, and germ cell tumors.
DB00338	https://go.drugbank.com/drugs/DB00338	Proton pump inhibitor used to treat GERD associated conditions such as heartburn and gastric acid hypersecretion, and to promote healing of tissue damage and ulcers caused by gastric acid and H. pylori infection.

Example- Parent Triples Maps with Join (2/4)

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
<TriplesMapDBDrug>
    rr:logicalTable [ rr:sqlQuery """SELECT DISTINCT ID, DBLink, DBSummary
                                                    FROM DBDrug """];
    rr:subjectMap [rr:template "http://example.org/DBDrug/{ID}";
                   rr:class ex:DBDrug];

    rr:predicateObjectMap [
        rr:predicate ex:hasDBLink;
        rr:objectMap [
            rr:column "DBLink";  rr:termType rr:IRI]];
    rr:predicateObjectMap [
        rr:predicate ex:summary;
        rr:objectMap [
            rr:column "DBSummary"]].
```

Example- Parent Triples Maps with Join (2/4)

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
<TriplesMapDBDrug>
    rr:logicalTable [ rr:sqlQuery """SELECT DISTINCT ID, DBLink, DBSummary
                                                    FROM DBDrug """];
    rr:subjectMap [rr:template "http://example.org/DBDrug/{ID}";
                   rr:class ex:DBDrug];

    rr:predicateObjectMap [
        rr:predicate ex:hasDBLink;
        rr:objectMap [
            rr:column "DBLink";  rr:termType rr:IRI]];
    rr:predicateObjectMap [
        rr:predicate ex:summary;
        rr:objectMap [
            rr:column "DBSummary"]].
```

Explanation:

- Defines a triples map for instances of the class `ex:DBDrug`.
- Uses the `rr:subjectMap` to uniquely identify instances with `ID`.
- `ex:hasDBLink` associates a URI from the `DBLink` column, treated as an IRI.
- `ex:summary` maps the `DBSummary` attribute as a literal value.

Example- Parent Triples Maps with Join (3/4)

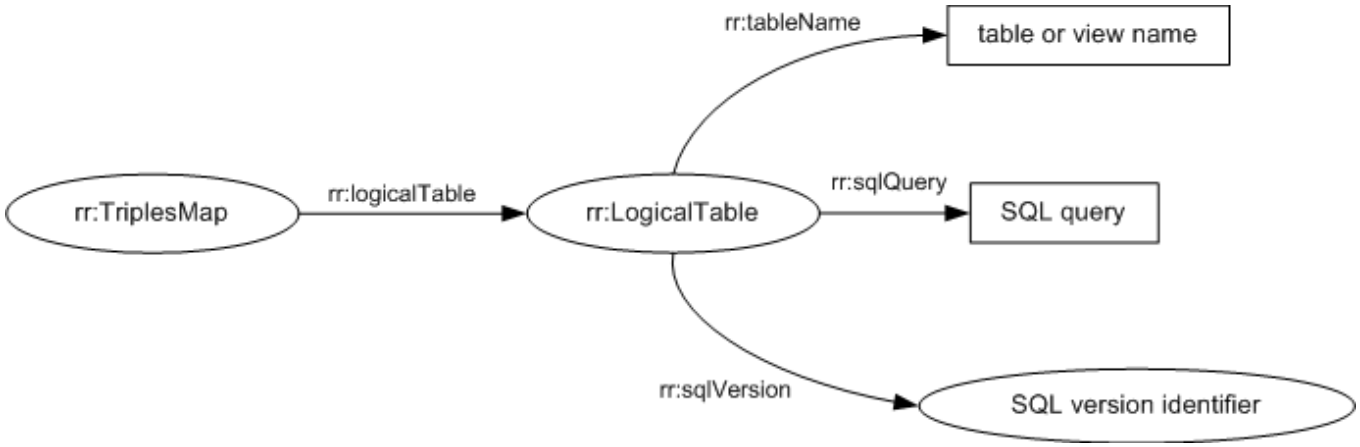
```
<TriplesMapDrug>
    rr:logicalTable [ rr:sqlQuery """SELECT DISTINCT drugBankID, DrugName,
                                Bioavailability, casNumber FROM Drug """];
 rr:subjectMap [rr:template "http://example.org/Drug/{drugBankID}";
                   rr:class ex:Drug];
 rr:predicateObjectMap [
  rr:predicate ex:drugName;
  rr:objectMap [
   rr:template "http://example.org/Drug/{DrugName}";]];
       rr:predicateObjectMap [
  rr:predicate ex:bioavailability;
  rr:objectMap [
   rr:column "Bioavailability"; rr:datatype xsd:float]];
      rr:predicateObjectMap [
  rr:predicate ex:casNumber;
  rr:objectMap [
   rr:column "casNumber"; rr:datatype xsd:string];
      rr:predicateObjectMap [
  rr:predicate ex:hasDBId;
  rr:objectMap [
   rr:parentTriplesMap <TriplesMapDBDrug>;
                    rr:joinCondition [rr:child "drugBankID";
                                      rr:parent "ID"]]]].
```

Example- Parent Triples Maps with Join (4/4)

Explanation:

- This example illustrates the use of a referencing object map to link the subject map in `TriplesMapDrug` with the subject map in `TriplesMapDBDrug`.
- The `rr:joinCondition` ensures that `drugBankID` in `TriplesMapDrug` matches `ID` in `TriplesMapDBDrug`.

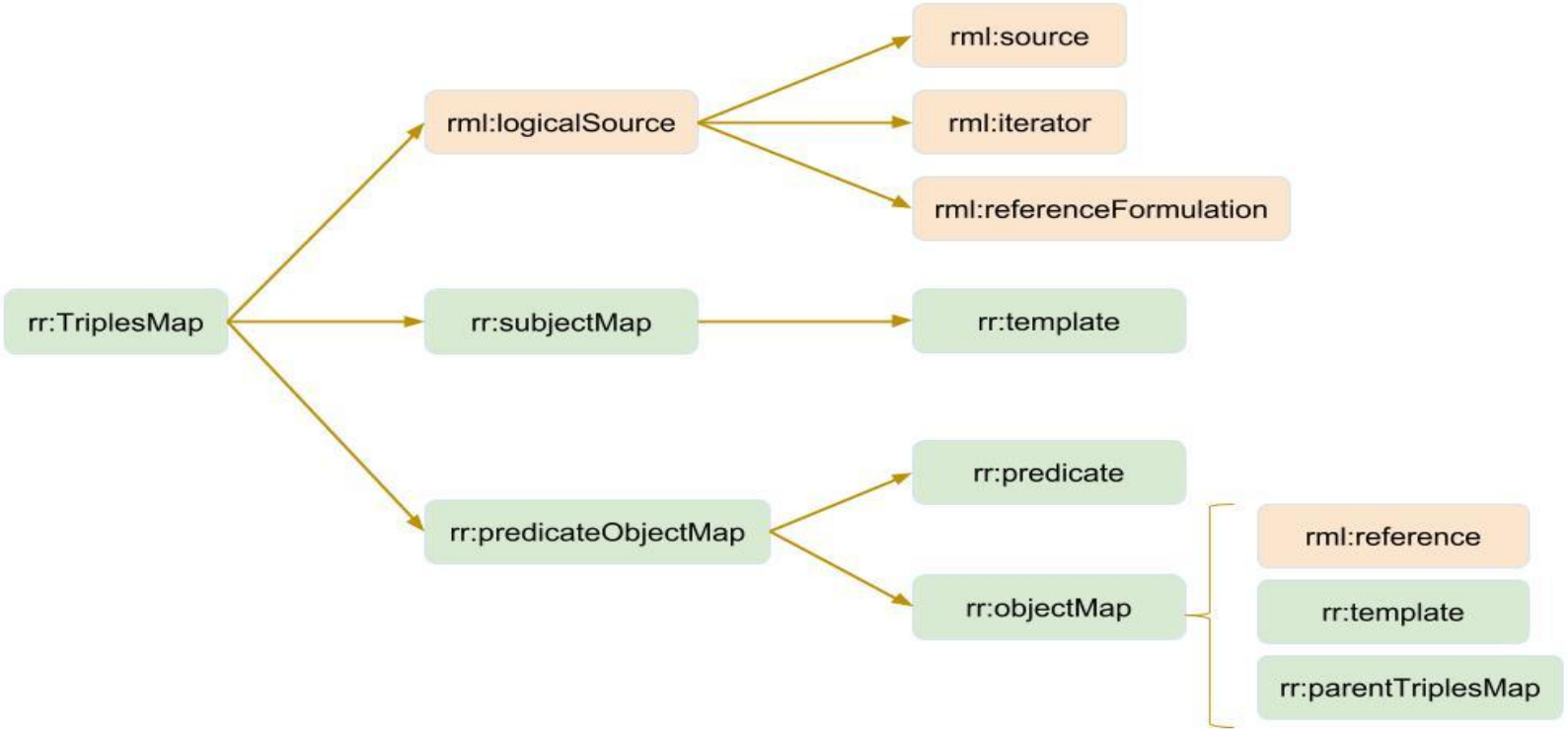
R2RML - Summary TriplesMaps in R2RML



- The `rr:logicalTable` defines the logical data source for a triples map.
- It connects to:
  - `rr:tableName`: Specifies the name of the table or view.
  - `rr:sqlQuery`: Allows for custom SQL queries over the data source.
  - `rr:sqlVersion`: Indicates the SQL version used.
- This structure ensures the mapping of relational tables into RDF triples.

RML

- The RDF Mapping Language (RML) is an extension of R2RML to enable the definition of data sources in various formats, i.e., XML, JSON, CSV, and relational tables. RML is expressed in RDF.



Main differences between R2RML and RML?

- **R2RML** (`https://www.w3.org/TR/r2rml/`) only allows for defining mappings between concepts in O and data sources that are relational tables.
- **RML** (`https://rml.io/specs/rml/`) is an extension of R2RML and allows for the definition of mappings between concepts in O and data sources in S in the formats CSV, XML, JSON, or relational databases.

Main differences between R2RML and RML?

- **R2RML** (`https://www.w3.org/TR/r2rml/`) only allows for defining mappings between concepts in *O* and data sources that are relational tables.
- **RML** (`https://rml.io/specs/rml/`) is an extension of R2RML and allows for the definition of mappings between concepts in *O* and data sources in *S* in the formats CSV, XML, JSON, or relational databases.

RML reuses concepts from R2RML and adds the following predicates:

- `rml:logicalSource` to specify a source.
- `rml:source` to point to a data source.
- `rml:reference` to define the object values of a data type property using attributes of the logical source.
- `rml:referenceFormulation` to specify the format of the logical data source.
- `rml:iterator` to indicate how a JSON or XML file can be iterated.

Comparison of R2RML and RML

R2RML	RML
Logical Table - only relational database (**rr:logicalTable**)	Logical Source - CSV, XML, JSON, HTML (**rml:logicalSource**)
Table Name (**rr:tableName**)	URI pointing to the source (**rml:source**) it can be RDB, JSON, XML, or CSV
Relational Table Column (**rr:column**)	Reference (**rml:reference**)
SQL query (**rr:sqlQuery**)	Reference Formulation (**rml:referenceFormulation**) - type of the source of the input data file, e.g. CSV, JSONPath, XPath.
Iteration per row in table	Definition of an iterator over JSON and XML (**rml:iterator**)

RML Example - Data Source in CSV

Suppose the data source with the drug data is in CSV format:

Drug.csv

DrugName	Bioavailability	casNumber	drugBankID
Vinorelbine	43.00	71486-22-1	DB00361
Cisplatin	100.00	15663-27-1	DB00515
Omeprazole	35.00	73590-58-6	DB00338

Data Source from where data is collected

```
<TriplesMapDrug>
  rml:logicalSource [
    rml:source "Drug.csv";
    rml:referenceFormulation ql:CSV
];
```

RML Example - Data Source in CSV

Suppose the data source with the drug data is in CSV format:

Drug.csv

DrugName	Bioavailability	casNumber	drugBankID
Vinorelbine	43.00	71486-22-1	DB00361
Cisplatin	100.00	15663-27-1	DB00515
Omeprazole	35.00	73590-58-6	DB00338

Data Source from where data is collected

```
<TriplesMapDrug>
  rml:logicalSource [
    rml:source "Drug.csv";
    rml:referenceFormulation ql:CSV
];
```

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .

<TriplesMapDrug>
    rml:logicalSource [
        rml:source "Drug.csv";
        rml:referenceFormulation ql:CSV
    ];
    rr:subjectMap [
        rr:template "http://example.org/Drug/{drugBankID}";
        rr:class ex:Drug
    ];
    rr:predicateObjectMap [
        rr:predicate ex:drugName;
        rr:objectMap [
            rr:template "http://example.org/Drug/{DrugName}"
        ]];
    rr:predicateObjectMap [
        rr:predicate ex:hasDBId;
        rr:objectMap [
            rr:template "http://example.org/Drug/{drugBankID}"
        ]];
    rr:predicateObjectMap [
        rr:predicate ex:bioavailability;
        rr:objectMap [
            rml:reference "Bioavailability";
            rr:datatype xsd:float
        ]];
    rr:predicateObjectMap [
        rr:predicate ex:casNumber;
        rr:objectMap [
            rml:reference "casNumber";
            rr:datatype xsd:string
        ]].
```

RML Example - Data Source in RDB

Suppose the data source with the drug data is a relational table

Drug

DrugName	Bioavailability	casNumber	drugBankID
Vinorelbine	43.00	71486-22-1	DB00361
Cisplatin	100.00	15663-27-1	DB00515
Omeprazole	35.00	73590-58-6	DB00338

Data Source from where data is collected

```
<TriplesMapDrug>
    rml:logicalSource
    [ rml:source <RDB_source>;
              rr:sqlVersion rr:SQL2008;
               rml:query """SELECT DISTINCT drugBankID,
               DrugName, Bioavailability,
               casNumber FROM Drug """]];
```

Define access to the database

```
<RDB_source> a d2rq:Database;
        d2rq:jdbcDSN "Example-RelationalTable";
        d2rq:jdbcDriver "com.mysql.cj.jdbc.Driver";
        d2rq:username "USER ";
        d2rq:password "PASS ".
```

RML Example - Data Source in RDB

Suppose the data source with the drug data is a relational table

Drug

DrugName	Bioavailability	casNumber	drugBankID
Vinorelbine	43.00	71486-22-1	DB00361
Cisplatin	100.00	15663-27-1	DB00515
Omeprazole	35.00	73590-58-6	DB00338

Data Source from where data is collected

```
<TriplesMapDrug>
    rml:logicalSource
    [ rml:source <RDB_source>;
              rr:sqlVersion rr:SQL2008;
               rml:query """SELECT DISTINCT drugBankID,
               DrugName, Bioavailability,
               casNumber FROM Drug """]];
```

Define access to the database

```
<RDB_source> a d2rq:Database;
        d2rq:jdbcDSN "Example-RelationalTable";
        d2rq:jdbcDriver "com.mysql.cj.jdbc.Driver";
        d2rq:username "USER ";
        d2rq:password "PASS ".
```

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
@prefix d2rq:<http://www.wiwiss.fu-berlin.de/suhl/bizer/D2RQ/0.
<TriplesMapDrug>
    rml:logicalSource [ rml:source <RDB_source>;
            rr:sqlVersion rr:SQL2008;
            rml:query """SELECT DISTINCT drugBankID, DrugName,
    rr:subjectMap [
         rr:template "http://example.org/Drug/{drugBankID}";
         rr:class ex:Drug];
    rr:predicateObjectMap [
         rr:predicate ex:drugName;
         rr:objectMap [
            rr:template "http://example.org/Drug/{DrugName}";
            ]];
    rr:predicateObjectMap [
         rr:predicate ex:hasDBId;
         rr:objectMap [
            rr:template "http://example.org/Drug/{drugBankID}";
            ]];
    rr:predicateObjectMap [
         rr:predicate ex:bioavailability;
         rr:objectMap [
            rml:reference "Bioavailability";
            rr:datatype xsd:float ]];
    rr:predicateObjectMap [
         rr:predicate ex:casNumber;
         rr:objectMap [
            rml:reference "casNumber";
            rr:datatype xsd:string ]].
<RDB_source> a d2rq:Database;
                d2rq:jdbcDSN "Example-RelationalTable";
                d2rq:jdbcDriver "com.mysql.cj.jdbc.Driver";
                d2rq:username "USER ";
                d2rq:password "PASS ".
```

Exercise

Participate **in the survey at Stud.IP**.

Example 1 - RML Triples Maps

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
<Production_CompanyTriplesMap>
      rml:logicalSource [rml:source "Movie_PC.csv";
                         rml:referenceFormulation ql:CSV];
      rr:subjectMap     [rr:template "http://example.org/{Company}";
                         rr:class ex:Production_Company].
```

Movie_PC.csv

Movie	Company
Shrek3	PC1
Shrek3	PC2

Example 1 - RML Triples Maps

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
<Production_CompanyTriplesMap>
       rml:logicalSource [rml:source "Movie_PC.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap     [rr:template "http://example.org/{Company}";
                          rr:class ex:Production_Company].
```

Movie_PC.csv

Movie	Company
Shrek3	PC1
Shrek3	PC2

What does the triples map `<Production_CompanyTriplesMap>` express?

- The triples map named `<Production_CompanyTriplesMap>` states the definition of the identifiers of the entities in the class `ex:Production_Company`. These identifiers are created concatenating "http://example.org/" with the values of the attribute Company from table Movie_PC.csv.

Example 1 - RML Triples Maps

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
<Production_CompanyTriplesMap>
       rml:logicalSource [rml:source "Movie_PC.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap     [rr:template "http://example.org/{Company}";
                          rr:class ex:Production_Company].
```

Movie_PC.csv

Movie	Company
Shrek3	PC1
Shrek3	PC2

What does the triples map `<Production_CompanyTriplesMap>` express?

- The triples map named `<Production_CompanyTriplesMap>` states the definition of the identifiers of the entities in the class `ex:Production_Company`. These identifiers are created concatenating "http://example.org/" with the values of the attribute Company from table Movie_PC.csv.

Which are the RDF triples generated by `<Production_CompanyTriplesMap>` triplesmap?

- The execution of the triples map `<Production_CompanyTriplesMap>` on the file Movie_PC.csv creates two RDF triples.

```
<http://example.org/PC1> a ex:Production_Company .
<http://example.org/PC2> a ex:Production_Company .
```

Example 2 - RML Triples Maps

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
<Production_CompanyTriplesMap>
       rml:logicalSource [rml:source "Movie_PC.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap     [rr:template "http://example.org/{Company}";
                          rr:class ex:Production_Company].
<MovieTriplesMap>
       rml:logicalSource [rml:source "Movie_PC.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap     [rr:template "http://example.org/{Movie}";
                          rr:class ex:Movie];
       rr:predicateObjectMap [
              rr:predicate ex:producedBy;
              rr:objectMap [
                rr:parentTriplesMap <Production_CompanyTriplesMap>]].
```

Movie_PC.csv

Movie	Company
Shrek3	PC1
Shrek3	PC2

Example 2 - RML Triples Maps

What does the triples map <MovieTriplesMap> express?

- The triples map named <MovieTriplesMap> states the definition of the identifiers of the entities in the class ex:Movie and the object values of the object property ex:producedBy. These values correspond to entities in class `ex:Production_Company` created in the execution of the triples map named <Production_CompanyTriplesMap>.

Example 2 - RML Triples Maps

What does the triples map <MovieTriplesMap> express?

- The triples map named <MovieTriplesMap> states the definition of the identifiers of the entities in the class ex:Movie and the object values of the object property ex:producedBy. These values correspond to entities in class ex:Production_Company created in the execution of the triples map named <Production_CompanyTriplesMap>.

Which are the RDF triples generated when <MovieTriplesMap> and <Production_CompanyTriplesMap> are executed?

- ```
  <http://example.org/PC1> a ex:Production_Company .
  <http://example.org/PC2> a ex:Production_Company .
  <http://example.org/Shrek3> a ex:Movie ;
          ex:producedBy <http://example.org/PC1> ,
                        <http://example.org/PC2> .
  ```
  ```

Example 3 - RML Triples Maps

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
<DirectorTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Director}";
                         rr:class ex:Director].
<Production_CompanyTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Company}";
                         rr:class ex:Production_Company].
<MovieTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Movie_ID}";
                         rr:class ex:Movie].
```

Director_PC_Movie.csv

Director	Movie_ID	Company
Chris Miller	Shrek3	PC1

Example 3 - RML Triples Maps

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
<DirectorTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Director}";
                         rr:class ex:Director].
<Production_CompanyTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Company}";
                         rr:class ex:Production_Company].
<MovieTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Movie_ID}";
                         rr:class ex:Movie].
```

Director_PC_Movie.csv

Director	Movie_ID	Company
Chris Miller	Shrek3	PC1

What do the triples maps <DirectorTriplesMap>, `<Production_CompanyTriplesMap>`, and `<MovieTriplesMap>` express?

- These triples maps state the definition of the identifiers of the entities in the classes `ex:Director`, `ex:Production_Company`, and `ex:Movie`.

Example 3 - RML Triples Maps

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ql <http://semweb.mmlab.be/ns/ql#> .
<DirectorTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Director}";
                         rr:class ex:Director].
<Production_CompanyTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Company}";
                         rr:class ex:Production_Company].
<MovieTriplesMap>
       rml:logicalSource [rml:source "Director_PC_Movie.csv";
                          rml:referenceFormulation ql:CSV];
       rr:subjectMap    [rr:template "http://example.org/{Movie_ID}";
                         rr:class ex:Movie].
```

Director_PC_Movie.csv

Director	Movie_ID	Company
Chris Miller	Shrek3	PC1

What do the triples maps <DirectorTriplesMap>, <Production_CompanyTriplesMap>, and <MovieTriplesMap> express?

- These triples maps state the definition of the identifiers of the entities in the classes ex:Director, ex:Production_Company, and ex:Movie.

Which are the RDF triples generated when <DirectorTriplesMap>, <Production_CompanyTriplesMap>, and <MovieTriplesMap> are executed?

- <http://example.org/ChrisMiller> a ex:Director .
  <http://example.org/PC1> a ex:Production_Company .
  <http://example.org/Shrek3> a ex:Movie .

SPARQL Queries for Mapping Definitions (1/3)

Classes' Definition

```
PREFIX rml: <http://semweb.mmlab.be/ns/rml#>
PREFIX rr: <http://www.w3.org/ns/r2rml#>

SELECT DISTINCT ?class ?typeDefinition ?source WHERE {
  ?triplesmap rml:logicalSource ?o .
  ?triplesmap rr:subjectMap ?o2 .
  ?o2 rr:class ?class .
}
```

Run the SPARQL queries on the SPARQL endpoint:
`https://labs.tib.eu/sdm/clarify-kg-mappings/sparql`

SPARQL Queries for Mapping Definitions (2/3)

Predicates' Definition

```
PREFIX rml: <http://semweb.mmlab.be/ns/rml#>
PREFIX rr: <http://www.w3.org/ns/r2rml#>

SELECT DISTINCT ?class ?propertyDefinition ?objectValue WHERE {
  ?triplesmap rml:logicalSource ?o .
  ?triplesmap rr:subjectMap ?o2 .
  ?o2 rr:class ?class .
  ?triplesmap rr:predicateObjectMap ?o4 .
  ?o4 rr:predicate ?property .
  ?o4 rr:objectMap ?o6 .
  ?o6 ?propertyDefinition ?objectValue .
}
```

Run the SPARQL queries on the SPARQL endpoint:

`https://labs.tib.eu/sdm/clarify-kg-mappings/sparql`

SPARQL Queries for Mapping Definitions (3/3)

Number Mappings Per Class

```
PREFIX rml: <http://semweb.mmlab.be/ns/rml#>
PREFIX rr: <http://www.w3.org/ns/r2rml#>

SELECT ?class count(DISTINCT ?triplesmap) as ?numberMappings WHERE {
  ?triplesmap rml:logicalSource ?o .
  ?triplesmap rr:subjectMap ?o2 .
  ?o2 rr:class ?class .
}
GROUP BY ?class ORDER BY DESC(?numberMappings)
```

Run the SPARQL queries on the SPARQL endpoint:
`https://labs.tib.eu/sdm/clarify-kg-mappings/sparql`

Summary & Conclusions (1/2)

What We Learned

- **Data Integration** enables uniform access to heterogeneous and autonomous data sources.
- **Declarative mapping languages** (R2RML, RML) specify *rules*, not code, to transform data into RDF.
- Mappings rely on a clear separation between:
  - **Sources** (CSV, JSON, relational tables),
  - **Ontology** (global conceptual view),
  - **Mappings** (alignment rules).

Summary & Conclusions (2/2)

Mapping Assertions

- **Concept mappings:** create class instances from source rows.
- **Role mappings:** create object properties
  - single-source,
  - multi-source (with joins),
  - referenced-source mappings.
- **Attribute mappings:** attach literal values to generated IRIs.

Summary & Conclusions (2/2)

Mapping Assertions

- **Concept mappings:** create class instances from source rows.
- **Role mappings:** create object properties
  - single-source,
  - multi-source (with joins),
  - referenced-source mappings.
- **Attribute mappings:** attach literal values to generated IRIs.

Key Takeaways

- Mappings are **declarative, reusable, and ontology-driven**.
- One source row can generate:
  - a new IRI,
  - links to other entities,
  - and literal attributes.
- Results: a **queryable, interoperable, and reasoning-ready KG**.

References I

A. Doan, A. Y. Halevy, and Z. G. Ives.
Principles of Data Integration.
Morgan Kaufmann, 2012.

M. Lenzerini.
Data integration: A theoretical perspective.
In L. Popa, S. Abiteboul, and P. G. Kolaitis, editors, *Proceedings of the Twenty-first ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 3-5, Madison, Wisconsin, USA*, pages 233–246. ACM, 2002.